

Harnessing Generalist Agents for Contextualized Time Series

Zihao Li, Kaifeng Jin, Yuanchen Bei, Jiaru Zou, Avaneesh Kumar, Xuying Ning
YanJun Zhao, Mengting Ai, Baoyu Jing, Hanghang Tong, Jingrui He

University of Illinois Urbana-Champaign

Correspondence: zihao15@illinois.edu, jingrui@illinois.edu

Abstract

Time series are often embedded in rich contexts that are essential for holistic modeling. Moreover, real-world practitioners often require end-to-end workflows for analyzing temporal dynamics, where widely studied tasks such as forecasting are only one step in a broader solution loop. While generalist AI agents offer a promising interface for such workflows under complex contexts, they still operate primarily in textual spaces that are not fully aligned with structured temporal signals. In this work, we introduce TIMECLAW, an agentic harness framework for time series that equips generalist LLM agents with the time-series-native runtime support needed for contextualized temporal reasoning. TIMECLAW integrates executable temporal tools for grounded and auditable analysis, experience-driven capability evolution for creating reusable analytical routines, and episodic multimodal memory for retrieving relevant reasoning traces. Together, these components unlock harnessed open-ended temporal reasoning with contextual information. Extensive evaluation on multiple benchmarks covering diverse tasks across energy, finance, weather, traffic, and other real-world domains demonstrates improved performance of TIMECLAW. Code is available at this [repository hyperlink](#).

1 Introduction

Time series provide a fundamental lens for observing complex real-world system dynamics and supporting decisions over time (Jiang et al., 2025; Wu et al., 2023; Goswami et al., 2024). Financial time series capture evolving market and macroeconomic conditions, guiding investment and risk-management decisions (Arsenault et al., 2025). Healthcare time series reflect the dynamic state of the human body, enabling early detection of potential illnesses and personalized treatment planning (Jarrett et al., 2021; Choi et al., 2016). Recently, the rapid development of large language models

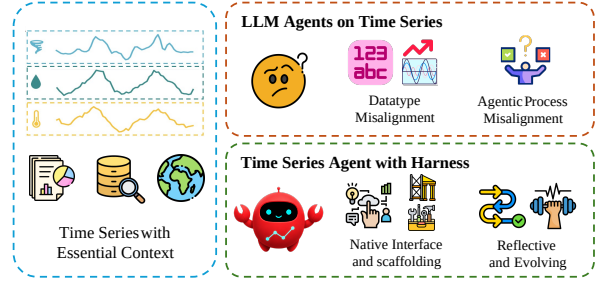


Figure 1: Contextualized time series require more than task-specific models or directly prompted LLM agents. TIMECLAW provides a time-series-native agent harness that leverages context to support holistic modeling.

(LLMs) and agentic AI offers new opportunities to move beyond conventional time-series modeling toward more holistic temporal understanding and reasoning (Chang et al., 2025b; Li et al., 2026a). This opportunity arises from two practical needs. First, most existing methods model time series as standalone numerical sequences, whereas real-world temporal signals are often accompanied by external contexts that are crucial for holistic modeling and can be explicitly leveraged by agents (Li et al., 2025c; Chang et al., 2026). Second, existing studies largely focus on predefined tasks, with forecasting being the most representative (Goswami et al., 2024), whereas real-world applications often require broader solution loops where such tasks are only intermediate steps (Jin et al., 2024c; Yu et al., 2026).

However, the generality of LLMs alone does not provide the scaffold needed for reliable time-series reasoning (Tan et al., 2024; Merrill et al., 2024). Directly inserting time series data as ordinary text for the LLM agents remains insufficient and causes two misalignments. The first is *datatype misalignment*: temporal signals encode statistical properties such as trends and periodicity that can be distorted when serialized into tokens and processed through language-oriented attention mechanisms (Jin et al., 2024a; Schwartz et al., 2024). The

second is *agentic process misalignment*: existing agentic workflows are primarily designed for text-centric information processing (Yao et al., 2023). Long numerical sequences may further burden the context window and distract the agent from reliable temporal reasoning (Tan et al., 2024).

To address such misalignments, agent harnesses have emerged as an effective way to equip LLM agents with reliable interfaces for perceiving and acting over non-text modalities (Meng et al., 2026; Ning et al., 2026b). Yet existing harness designs rarely account for the unique structure of time series. This motivates our central research question:

How to build time-series-native agent harness?

To answer this question, we introduce TIMECLAW, an agentic harness framework for contextualized time-series understanding, reasoning, and decision-making. Specifically, TIMECLAW integrates executable temporal tools, experience-driven capability evolution, and episodic multimodal memory. The executable temporal tools enable agents to inspect, transform, analyze, and act upon time series as structured temporal objects through auditable workflows. To address datatype misalignment, TIMECLAW operates on time-series-native runtime instead of serialized text. The experience-driven capability evolution mechanism allows the agent to expand its temporal reasoning abilities over time and address agentic process misalignment. The episodic multimodal memory module retrieves relevant past reasoning traces to guide new tasks with similar temporal patterns or contexts. Extensive experiments across diverse benchmarks and real-world domains show that TIMECLAW consistently improves end-to-end performance while maintaining token efficiency. Our contributions are summarized as follows:

- We identify the limitations of directly using LLMs for time-series reasoning, showing that textualizing temporal signals induces datatype and agentic-process misalignments.
- We introduce TIMECLAW, a time-series-native agentic harness that integrates executable tools, experience-driven capability evolution, and episodic multimodal memory.
- We conduct extensive experiments across diverse real-world domains, demonstrating the effectiveness of TIMECLAW for end-to-end reasoning over contextualized time series.

2 Background

2.1 Preliminary

We use calligraphic letters (e.g., $\mathcal{A}$) for sets and bold capital letters for matrices (e.g., $\mathbf{A}$). For matrix indices, $\mathbf{A}[i, j]$ denotes the entry in the i^{th} row and the j^{th} column. For a vector $\mathbf{v}$, $\mathbf{v}[i : j]$ represents the sub-vector sliced from the i^{th} to the j^{th} position, inclusively. $\mathbf{A}[i, :]$ returns the i^{th} row in $\mathbf{A}$ and $\mathbf{A}[: i]$ returns the first i rows of $\mathbf{A}$.

Time Series and Contextualized Time Series. A *numerical time series* is denoted as

$$\mathbf{X}_{1:T} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T] \in \mathbb{R}^{N \times T}, \quad (1)$$

where T is the number of timestamps, N is the number of variables, and $\mathbf{x}_t \in \mathbb{R}^N$ is the observation at timestamp t . For $1 \leq a \leq b \leq T$, we denote the temporal slice from timestamp a to b as

$$\mathbf{X}_{a:b} = [\mathbf{x}_a, \mathbf{x}_{a+1}, \dots, \mathbf{x}_b]. \quad (2)$$

We define a *contextualized time series* as

$$\mathcal{X} = (\mathbf{X}_{1:T}, \mathcal{C}), \quad (3)$$

where $\mathbf{X}_{1:T}$ is the numerical temporal signal and $\mathcal{C}$ is the associated context. The context $\mathcal{C}$ may contain information from various sources, such as textual descriptions, categorical metadata, external constraints, and domain knowledge. In this work, we focus on the text-time series mixture, a special case of contextualized time series. This setting is broadly representative of many real-world temporal reasoning scenarios where textual cues provide essential information for interpreting temporal dynamics and supporting decisions. Yet the core methodology is generalizable across additional context sources beyond natural language.

From Predefined Tasks to Open-Ended Temporal Reasoning. Conventional time-series learning is usually defined by fixed task mappings, such as $\mathbf{X}_{1:T} \mapsto \mathbf{X}_{T+1:T+H}$ for forecasting or $\mathbf{X}_{1:T} \mapsto y$ for classification. We define *open-ended contextualized temporal reasoning* as a more general setting where the task objective and output format are specified by a natural-language instruction, and the numerical time series is interpreted together with its associated context. Given a contextualized time series $\mathcal{X} = (\mathbf{X}_{1:T}, \mathcal{C})$, we define a task instance as

$$\tau = (q, \mathcal{X}, \mathcal{Y}, y^*, \ell), \quad (4)$$

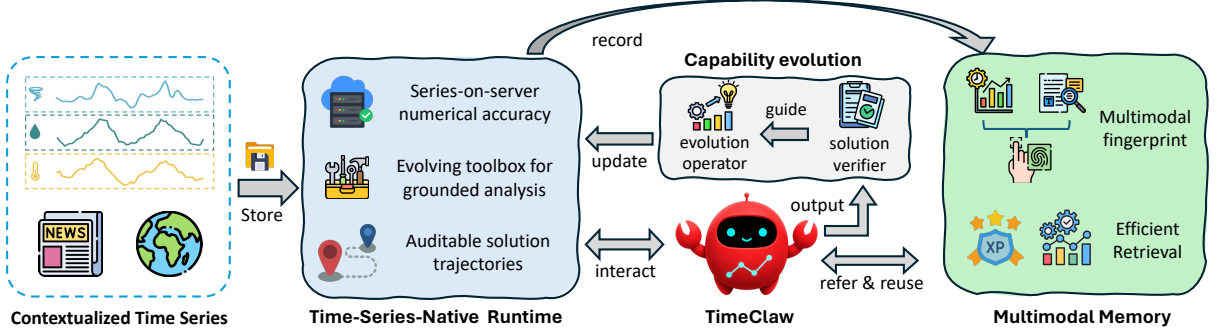


Figure 2: Overview of TIMECLAW. Given contextualized time series, TIMECLAW operates through a time-series-native runtime that provides server-side numerical execution, an evolving toolbox for grounded analysis, and auditable solution trajectories (Section 3.2). The agent further improves over time through capability evolution (Section 3.3) and retrieves relevant experience from multimodal memory via multimodal fingerprints (Section 3.4).

where q is the instruction, $\mathcal{Y}$ is the output space, $y^* \in \mathcal{Y}$ is the target, and $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$ is the evaluation function. The goal is to produce

$$\hat{y} = f(q, \mathcal{X}) \in \arg \min_{y \in \mathcal{Y}} \ell(y, y^*). \quad (5)$$

Here $\mathcal{Y}$ may include numerical predictions, labels, rankings, textual explanations, decisions, executable actions, or their combinations.

2.2 Why NOT Fully Trust LLMs?

LLM Workflow for Time Series. Existing LLM-based agents usually process contextualized time series through textual serialization. Given $\mathcal{X} = (\mathbf{X}_{1:T}, \mathcal{C})$, a serialization function σ maps the numerical series and context to text $s_{\mathcal{X}} = \sigma(\mathcal{X})$. The LLM then tokenizes the instruction and serialized input as

$$\mathbf{z}_{1:L} = \text{Tokenize}([q; s_{\mathcal{X}}]), \quad (6)$$

Limitations. This text-centric workflow introduces two forms of misalignment. First, *data-type misalignment* arises because numerical temporal signals are converted into discrete text tokens. As a result, numerical proximity, temporal resolution, long-range dependencies, and fine-grained variations may not be faithfully preserved under tokenization and finite context budgets. Second, *agent-process misalignment* arises because the agent reasons over serialized text rather than native temporal objects. Operations that are natural for time series, such as slicing, aggregation, decomposition, smoothing, forecasting, anomaly localization, and numerical verification, become indirect language-based reasoning steps. Due to page limitations, we provide a more detailed analysis of these limitations in Appendix A, including

numerical-token distance mismatch, decimal place-value distortion, next-token versus next-value objective mismatch, and long-context temporal information dilution.

3 Time-Series-Native Agent Harness

We address the misalignment between language agents and time-series tasks through an *agent harness* (Ning et al., 2026b): a time-series-native runtime that surrounds a frozen LLM policy and reshapes how it interacts with contextualized temporal data. Instead of forcing numerical signals into the token stream, the harness hosts temporal objects in a structured runtime environment and exposes them only through typed executable interfaces. The agent analyzes them through auditable temporal operations, while intermediate analytical states are maintained in a structured workspace.

3.1 Harness Framework Overview

TIMECLAW instantiates this harness through three coordinated components. *Executable temporal tools* (Section 3.2) provide typed interfaces for accessing and analyzing time-series objects, producing grounded and auditable analytical evidence. *Experience-driven capability evolution* (Section 3.3) abstracts recurring procedures as useful temporal routines to be reused rather than repeatedly rediscovered. *Episodic multimodal memory* (Section 3.4) stores past reasoning traces and retrieves relevant experience through complementary textual and temporal modalities. Formally, TIMECLAW augments a frozen LLM policy π_{θ} with a harness $\mathcal{H}$:

$$\mathcal{H} = (\mathcal{T}, \mathcal{E}, \mathcal{M}), \quad (7)$$

where $\mathcal{T}$ is the set of executable temporal tools, $\mathcal{E}$ is the capability-evolution loop and $\mathcal{M}$ is the

episodic multimodal memory.

Workspace and rollout. For each task $\tau = (q, \mathcal{X}, \mathcal{Y}, y^*, \ell)$, $\mathbf{X}_{1:T}$ is loaded into a task-local *workspace* $\mathcal{W}$ hosted by the harness runtime that can be accessed through protocols (e.g., Model Context Protocol (Anthropic, 2024)). This workspace preserves the series at full numerical precision. The policy interacts with the harness through a trajectory

$$\zeta = (a_1, o_1, \dots, a_K, o_K, \hat{y}), \quad (8)$$

where each action $a_r \in \mathcal{A}$ denotes an agent decision, such as executing a runtime operation or performing a reasoning step. The corresponding observation $o_r \in \mathcal{O}$ records the outcome of this decision, either from the harness or the agent’s internal reasoning. The policy selects

$$a_r \sim \pi_\theta(\cdot \mid q, \mathcal{C}, o_{1:r-1}), \quad (9)$$

conditioned on the instruction, context, and accumulated evidence, ultimately emitting $\hat{y} \in \mathcal{Y}$.

3.2 Runtime-Native Temporal Tools

Despite differing in domains and objectives, time-series tasks often rely on a shared list of common temporal operations. Unlike conventional tool-augmented agents that must pass data through language messages, TIMECLAW executes these operations inside the harness runtime, where the full time-series object already resides in the workspace $\mathcal{W}$. Thus, the agent only specifies the intended operation and its parameters without the need to spend context budget on tool interaction. Each tool runs directly on $\mathcal{W}$ at numerical precision and returns a compact structured observation. We initialize $\mathcal{T}$ with common time series tools and allow new tools to emerge through capability evolution, as summarized in Table 10.

Auditable trajectories as reusable experience.

The harness requires every numerical claim to be grounded in a returned tool observation rather than inferred from free-form language reasoning. Each tool-related pair (a_r, o_r) in the rollout also records the analytical intent behind that invocation. The resulting evidence chain ties the final answer $\hat{y}$ to verifiable temporal operations and forms a reusable problem-solving trace, which further supports capability evolution and episodic memory.

3.3 Experience-Driven Capability Evolution

Runtime-native tools cover common temporal primitives, but real workflows often reveal recurring task-specific routines that cannot all be anticipated at design time. The second facet of the harness turns such repeated experience into executable capability. Specifically, an evolution operator $\mathcal{E}$ identifies verified recurring sub-procedures and admits them into the tool set $\mathcal{T}$. As a result, $\mathcal{T}$ evolves and expands with the target task distribution.

Capability evolution mechanism. We implement the evolution operator $\mathcal{E}$ as a code-specialized LLM. Given clustered trajectories and task descriptions, $\mathcal{E}$ abstracts recurring analytical sub-procedures into self-contained executable tools, each with an invocation docstring and an implementation that operates on the runtime workspace $\mathcal{W}$. Each candidate tool is verified by execution on a held-out subset of similar tasks and is admitted into $\mathcal{T}$ only if its held-out success rate exceeds a threshold γ . Once admitted, the tool is immediately registered alongside the seed tools and can be invoked in future rollouts.

3.4 Episodic Multimodal Memory

Episodic memory is common in agentic systems, but its design is nontrivial in contextualized time-series settings, since relevance depends on both task framing and temporal structure. A text-based or jointly serialized retrieval space can (1) retrieve misleading precedents, since token similarity does not necessarily preserve numerical proximity (Proposition 1); (2) risk modality imbalance for long series; (3) exhibit scale instability, as normalization choices can change which temporal patterns appear similar.

Memory records. Each memory record $m \in \mathcal{M}$ stores a past task, its retrieval keys, and its successful analytical trace:

$$m = (\tau, \phi, \psi, \zeta, \rho), \quad (10)$$

where τ is the original task, ζ is the rollout trajectory produced under the harness, and ρ is a short transferable rationale summarizing the contextual cue and analytical rule behind the solution. For example, in the energy domain, ρ may note that a snowstorm scenario implies a sharp electricity demand increase due to air-conditioning load. The remaining fields ϕ and ψ serve as retrieval keys over the signal and text modalities, respectively.

Table 1: Overall results on the CiK benchmark. We report token usage when applicable. The best and second-best results in each metric column are highlighted in **bold** and underlined, respectively. TIMECLAW achieves the best average RCRPS and sMAPE, obtains the best RCRPS in 4 out of 5 context types, and is nearly tied regarding the remaining context type. Meanwhile, TIMECLAW achieves substantially better performance with nearly half the token budget compared with the second-best multi-agent solution.

Method	Average RCRPS ($\downarrow$)	Average sMAPE ($\downarrow$)	Average Token Usage ($\downarrow$)	RCRPS by Context Type ($\downarrow$)				
				Intemporal	Historical	Future	Covariate	Causal
Traditional Time Series Models								
ARIMA (1989)	0.2772	91.2514	—	0.2987	0.1032	0.1577	0.2073	0.2822
ETS (2017)	0.3282	84.5230	—	0.3654	0.1738	0.1966	0.2402	0.3290
DLinear (2023)	2.3448	184.1755	—	3.0988	4.9004	2.6461	1.9710	3.0394
PatchTST (2023)	2.3374	166.8354	—	3.0980	4.9007	2.6294	1.9614	3.0523
Time Series Foundation Models								
Chronos-2 (2025)	0.2344	90.8453	—	0.2277	0.1852	0.2262	0.2406	0.3426
Lag-Llama (2023a)	0.2628	95.1976	—	0.2475	0.1233	0.1987	0.2461	0.3299
Moirai-Large 2024a	0.4545	107.8152	—	0.3865	0.0929	0.3936	0.3792	0.2690
LLM/Agent on Time Series								
UniTime (2024b)	0.2822	93.3854	—	0.2910	<u>0.0889</u>	0.1665	0.2964	0.2677
Time-LLM (2024b)	0.3990	102.6825	—	0.3629	0.1144	0.3172	0.3589	0.3146
TS-Agent (2025b)	0.1421	61.2980	47455.5880	0.1434	0.0904	0.1210	0.1516	0.1596
TSci (2025a)	0.1448	69.9068	47905.1056	0.1540	0.1001	0.1181	0.1694	0.1971
General Agentic Pipelines								
Direct Prompt	0.1703	60.4407	24861.3451	0.1938	0.1244	0.1239	0.1880	0.2793
CoT Prompt (2022a)	0.1726	66.3724	27120.8662	0.1771	0.1230	0.1520	0.1932	0.2392
ReAct (2023)	0.1514	66.2284	25502.9507	0.1678	0.1452	0.1317	0.1681	0.2249
Self-Reflection (2024)	0.1608	64.7013	28942.8662	0.1869	0.1240	0.1131	0.1851	0.2756
Multi-Agent Reflection (2025)	<u>0.1294</u>	<u>55.5704</u>	63033.0070	<u>0.1346</u>	0.1086	<u>0.1108</u>	<u>0.1437</u>	0.1697
TIMECLAW (Ours)	0.1145	52.4542	35553.2113	0.1303	0.0886	0.0703	0.1239	<u>0.1597</u>

Multimodal retrieval keys. For holistic memory retrieval, we design retrieval keys for both modalities. The context key $\phi = \Phi(c_\tau)$ embeds a textual descriptor c_τ constructed from the task context. The time series key $\psi = \Psi(\mathbf{X}_{1:T})$ is a time-series fingerprint that encodes structural, statistical, spectral, temporal-dynamic, and multivariate properties. Due to page limitations, we provide details of fingerprint computation in Appendix B.

Similarity computation and retrieval. Given a query task τ_q with keys (ψ_q, ϕ_q) , the text-side similarity is defined by cosine similarity,

$$s_{\text{text}}(q, m) = \cos(\phi_q, \phi_m), \quad (11)$$

while the time series similarity is defined by the negative distance between normalized fingerprints,

$$s_{\text{ts}}(q, m) = -\|\tilde{\psi}_q - \tilde{\psi}_m\|_2, \quad (12)$$

where $\tilde{\psi}$ denotes the normalized fingerprint. These two scores can be merged in different ways, such as weighted aggregation, reciprocal-rank fusion, or staged retrieval. In TIMECLAW, we adopt a simple yet effective two-stage top-k retrieval strategy with details provided in Appendix C.

4 Experiments

We comprehensively evaluate TIMECLAW on three recent benchmarks spanning diverse application domains, demonstrating its effectiveness across a wide range of open-ended contextualized time-series scenarios.

Benchmarks and Evaluation Focus. We evaluate TIMECLAW on three complementary benchmarks. Context-is-Key (CiK) (Williams et al., 2025) examines context utilization in context-aware time-series reasoning; TSRBench (Yu et al., 2026) evaluates open-ended generalist capability through multi-task multimodal scenarios; and TSAIA (Ye et al., 2025) focuses on practical domain-specific financial analysis. We provide details in Appendix E.1

Baselines. We compare TIMECLAW with a broad set of baselines spanning five families. (a) *Traditional time-series models*, including statistical forecasting methods ARIMA and ETS (Shchur et al., 2023), as well as neural forecasting models DLinear (Zeng et al., 2023) and PatchTST (Nie et al., 2023). (b) *Time-series foundation models*, including Chronos-family models (Ansari et al., 2024a, 2025), Lag-Llama (Rasul et al., 2023a), and Moirai-family models (Woo et al., 2024a). (c) *LLMs and agents designed for time series*, in-

Table 2: Performance on TSRBench across open-ended temporal reasoning tasks. The best and second-best results in each metric column are highlighted in **bold** and underlined, respectively. All methods use GPT-5-nano as the base model. TIMECLAW achieves the best average accuracy while using the fewest tokens.

Method	Perception	Reasoning	Prediction	Decision Making	Avg. Accuracy (%)	Avg. Token Consumption
CoT (Wei et al., 2022b)	48.57	36.84	46.30	28.57	40.05	22,274
ReAct (Yao et al., 2023)	55.71	36.26	43.52	<u>31.75</u>	40.78	25,165
Self-Reflection (Renze and Guven, 2024)	50.00	<u>40.93</u>	38.89	30.16	40.29	25,799
Multi-Agent Reflection (Yuan and Xie, 2025)	62.85	33.92	<u>54.63</u>	25.40	<u>42.96</u>	45,279
TSci (Zhao et al., 2025a)	65.71	33.33	43.52	26.98	40.53	51,273
TIMECLAW (Ours)	<u>64.29</u>	43.86	59.26	33.33	49.76	15,979

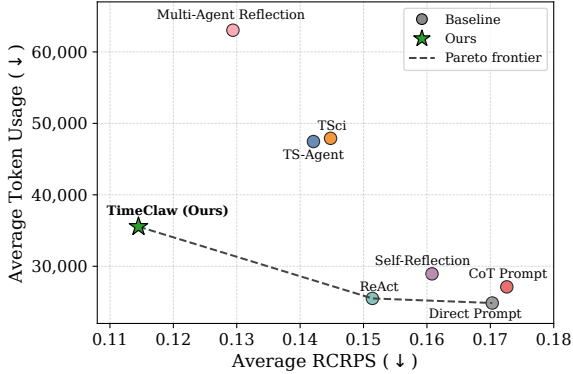


Figure 3: Performance-efficiency trade-off on CiK. Among methods with reported token usage, TIMECLAW achieves the best RCRPS with substantially lower token consumption than the strongest multi-agent baseline.

cluding UniTime (Liu et al., 2024b), Time-LLM (Jin et al., 2024b), TS-Agent (Liu et al., 2025b), and TSci (Zhao et al., 2025a). (d) *Open-source LLMs*, such as LLaMA3-70B (Meta, 2024). (e) *General agentic pipelines*, including direct prompting, chain-of-thought prompting (Wei et al., 2022a), ReAct (Yao et al., 2023), self-reflection (Renze and Guven, 2024), and multi-agent reflection (Yuan and Xie, 2025). Additional details are provided in Appendix G.

4.1 Context Matters for Time Series, and TIMECLAW Uses It More Effectively

Metrics. RCRPS is the primary metric of CiK. It emphasizes context-relevant regions and penalizes violations of task-specific constraints. We additionally report sMAPE as a widely used metric in time series. We also report token usage when applicable. Detailed metric definitions are provided in Appendix E.4.

Main Results. Table 1 summarizes the results on the CiK benchmark. Overall, methods that can access and reason over contextual information generally outperform context-agnostic models. DLin-

ear and PatchTST show large errors because they require the training length to exceed the prediction length, which can fail in open-ended tasks. TIMECLAW achieves the best average RCRPS and sMAPE, improving average RCRPS by 11.5% relative to the strongest baseline. TIMECLAW obtains the best RCRPS in 4 out of 5 context categories and is nearly tied with the best baseline in the remaining one. Notably, TIMECLAW achieves these gains efficiently. Compared with Multi-Agent Reflection, TIMECLAW uses 43.6% fewer tokens while achieving better average RCRPS and sMAPE. We provide additional domain-wise results in Table 12.

4.2 TIMECLAW Improves Open-Ended Temporal Reasoning

We next evaluate TIMECLAW on TSRBench (Yu et al., 2026), a generalist multi-task multimodal benchmark spanning perception, reasoning, prediction, and decision making. Numerical-only time-series models cannot handle end-to-end scenarios.

Table 2 compares TIMECLAW with representative agentic frameworks, all using GPT-5-nano as the base model. TIMECLAW achieves the best average accuracy with a 15.8% relative improvement while using the fewest tokens. Across task categories, TIMECLAW achieves the best performance except for ranking second on perception. This suggests that although serialization can encode sufficient information under a large context window, agents may still struggle to decode it into actionable insights. With agent harness, TIMECLAW yields strong gains on reasoning and prediction, where time-series-specific analysis is more critical.

We further compare TIMECLAW with various open-source models, including general LLMs, multimodal vision-language models (VLMs), and time-series LLMs (TSLLMs). Time series are represented as textual numerical sequences for LLMs, plots for VLMs, and projector-based embeddings

Table 3: TIMECLAW compared with open-source models on TSRBench. Baseline results are taken from the original TSRBench paper (Yu et al., 2026). By harnessing GPT-5-nano, TIMECLAW achieves the best performance, outperforming large open-source models with up to 235B parameters. Full results in Table 13.

Method	Percept	Reason	Predict	Decide	Overall
<i>General LLMs</i>					
Qwen2.5-3B	45.4	27.3	40.4	23.3	33.2
Qwen2.5-7B	49.6	28.3	34.8	28.5	33.7
Qwen2.5-72B	55.6	39.7	43.9	32.2	42.4
Qwen3-1.7B	47.0	27.7	48.7	28.2	36.6
Qwen3-8B	52.6	29.1	47.4	32.0	38.3
Qwen3-235B-A22B	63.8	40.9	35.6	32.5	42.1
Gemma3-12B-it	54.3	30.6	37.7	32.9	36.8
Gemma3-27B-it	57.0	33.7	38.3	31.8	38.6
<i>Multimodal LLMs</i>					
Phi4-Multimodal-8B	49.0	26.0	30.0	32.1	31.9
InternVL3.5-1B	45.9	27.8	38.6	28.2	33.8
InternVL3.5-8B	59.3	35.6	39.4	28.2	39.5
InternVL3.5-38B	60.3	38.1	32.3	32.3	39.5
MiniCPM-V-4.5-8B	61.3	26.5	39.0	27.9	35.9
MiMo-VL-7B-RL	59.1	33.1	33.2	30.9	37.2
<i>Time Series LLMs</i>					
OpenTSLM-tsqa-sp-3B	39.7	26.8	33.5	28.5	31.0
ChatTS-14B	50.7	29.3	30.2	28.5	33.0
TS-Reasoner-7B	52.1	31.3	39.5	27.6	36.4
TimeOmni-1-7B	54.7	31.0	36.5	32.2	36.6
TIMECLAW (Ours)	64.3	43.9	59.3	33.3	49.8

for TSLLMs. As shown in Table 3, TIMECLAW achieves the best performance across all task categories, outperforming larger open-source models with up to 235B parameters with GPT-5-nano.

4.3 TIMECLAW Applicable in Practice

We finally evaluate whether TIMECLAW remains effective in a practical domain-specific setting. We use TSAIA (Ye et al., 2025), a financial time-series analysis benchmark. In addition to general agentic baselines, we include three finance-specific agent systems: FinCon (Yu et al., 2024), FinTeam (Wu et al., 2025), and TradingAgents (Xiao et al., 2024b).

Main Results. Figure 4 reports the multiple-choice accuracy on TSAIA. Notably, through the capability-evolution loop in Section 3.3, TIMECLAW autonomously evolves three reusable routines from bank-building trajectories: `portfolio_var`, `portfolio_sharpe`, and `capm_regression`. Equipped with these evolved routines, TIMECLAW substantially outperforms all baselines by 38.9% relative improvement, demonstrating that TIMECLAW can acquire practical domain capabilities from experience.

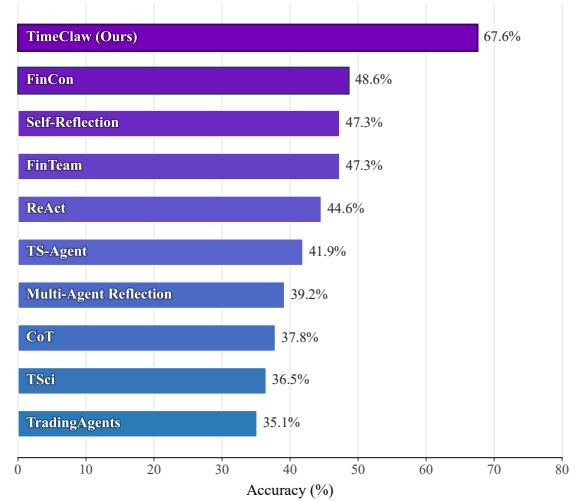


Figure 4: Performance on TSAIA. TIMECLAW substantially outperforms both general agentic baselines and finance-specific agents, demonstrating its practical applicability in financial time-series analysis.

4.4 Ablation Study

We conduct ablation studies to examine the effects of memory retrieval, backbone models, and framework components. As shown in Figure 5(a), enabling episodic memory retrieval improves both SMAPE and RCRPS, and larger retrieval sizes further improve performance. This shows that relevant past reasoning trajectories provide useful guidance for new contextualized time-series tasks. Figure 5(b) shows that TIMECLAW benefits from stronger LLM backbones, suggesting that time-series-native support complements general reasoning ability. Finally, Figure 5(c) shows that removing the tool harness, capability evolution, or memory all degrades performance, confirming that each part contributes to the final result. Due to page limitations, we provide further ablation studies in Appendix F, including memory bank size ablation and model family compatibility ablation.

4.5 Case Study

We provide detailed case studies in Appendix H to illustrate how TIMECLAW works on concrete tasks. These examples show how TIMECLAW leverages textual context under identical numerical histories, uses retrieved memory to guide causal reasoning, and evolves reusable tools from prior trajectories. They qualitatively demonstrate that the gains of TIMECLAW come from interpretable uses of context, memory, and experience-driven capability evolution.

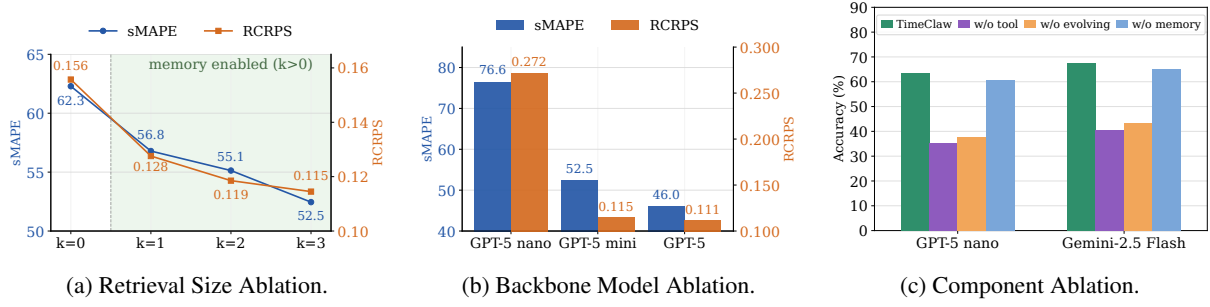


Figure 5: Ablation studies of TIMECLAW. (a) Retrieval size ablation shows the benefit of enabling memory retrieval. (b) Backbone model ablation shows that TIMECLAW consistently benefits from stronger LLM backbones. (c) Component ablation verifies the contributions of the tool harness, capability evolution, and memory.

5 Related Work

In this section, we review the key related works on the topics that are closely related to this work. We put more related works and discussions in the Appendix D to keep the main text concise.

Time-Series-only Analysis and Reasoning. Traditional time series analysis has predominantly addressed core tasks like forecasting, anomaly detection, and classification using specialized deep learning architectures (Zhou et al., 2021; Wu et al., 2021; Nie et al., 2023). Recently, inspired by the success of LLMs, time series foundation models tokenize time series patches and are trained in a similar way to LLMs (Jin et al., 2024a; Ansari et al., 2024b; Woo et al., 2024a). Beyond numerical sequences, some recent studies transform time series as images and leverage the reasoning abilities of Vision-Language Models (VLMs) to model time series (Zhong et al., 2025; He et al., 2026). TIMECLAW focuses on a more general setting of contextualized time-series reasoning, where temporal signals are analyzed with essential contexts.

Contextualized and Multimodal Time Series.

Real-world time series are heavily influenced by external events, prompting a shift to integrate external multimodal knowledge (Liu et al., 2025a; Kong et al., 2025b; Sun et al., 2024; Li et al., 2024). Recent frameworks demonstrate significant improvement in forecasting accuracy with contexts (King et al., 2023; Liu et al., 2024b). Building on these contextual advances, recent work has expanded from traditional numerical estimation to Time Series Question Answering (TSQA), enabling natural-language-driven description over complex temporal behaviors of the time series (Chen et al., 2025b; Wang et al., 2025; Jing et al., 2026; Kong et al.,

2025a). Yet, most existing work on contextualized time series still centers on predefined tasks such as forecasting. In contrast, TIMECLAW targets generalized end-to-end workflows that require agents to jointly interpret time series, analyze with context, and produce actionable solutions accordingly.

Agentic Systems for Time-Series Workflows.

As time series analysis grows in complexity, recent advancements have started applying agentic AI for time series (Wu et al., 2026; Liu et al., 2025b). While the broader AI literature explores agents across a wide array of domains, the application of LLM agents within time series research is currently overwhelmingly concentrated on forecasting tasks and time series property analysis tasks (e.g., periodicity, trend, stationary) (Chang et al., 2025a; Zhao et al., 2025a; Lee et al., 2025; Huang et al., 2026a). While these agentic workflows show potential in forecasting accuracy and temporal property analysis, their objectives remain largely prediction-centric. TIMECLAW takes a step toward a more reliable and general class of time-series agents that support context-rich and end-to-end workflow.

6 Conclusion

In this work, we introduce TIMECLAW, a time-series-native agentic harness for reasoning over contextualized time series. TIMECLAW equips generalist LLM agents with executable temporal tools, experience-driven capability evolution, and episodic multimodal memory. Across multiple time-series benchmarks, TIMECLAW consistently improves over both traditional and agentic LLM baselines. These empirical results demonstrate that harnessing generalist agents with native temporal interfaces is a promising direction for building reliable and adaptable time-series intelligence.

References

- Mengting Ai, Tianxin Wei, Sirui Chen, and Jingrui He. 2025a. Nirvana: Structured pruning reimaged for large language models compression. *arXiv preprint arXiv:2509.14230*.
- Mengting Ai, Tianxin Wei, Yifan Chen, Zhichen Zeng, Ritchie Zhao, Girish Varatkar, Bitu Darvish Rouhani, Xianfeng Tang, Hanghang Tong, and Jingrui He. 2025b. Resmoe: Space-efficient compression of mixture of experts llms via residual restoration. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, pages 1–12.
- Abdul Fatir Ansari, Oleksandr Shchur, Jaris Küken, Andreas Auer, Boran Han, Pedro Mercado, Syama Sundar Rangapuram, Huibin Shen, Lorenzo Stella, Xiyuan Zhang, Mononito Goswami, Shubham Kapoor, Danielle C. Maddix, Pablo Guerron, Tony Hu, Junming Yin, Nick Erickson, Prateek Mutalik Desai, Hao Wang, and 4 others. 2025. [Chronos-2: From univariate to universal forecasting](#). *CoRR*, abs/2510.15821.
- Abdul Fatir Ansari, Lorenzo Stella, Ali Caner Türkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda-Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Bernie Wang. 2024a. [Chronos: Learning the language of time series](#). *Trans. Mach. Learn. Res.*, 2024.
- Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, and 1 others. 2024b. [Chronos: Learning the language of time series](#). *Transactions on Machine Learning Research*.
- Anthropic. 2024. Model context protocol. <https://docs.anthropic.com/en/docs/agents-and-tools/mcp>.
- Pierre-Daniel Arsenault, Shengrui Wang, and Jean-Marc Patenaude. 2025. [A survey of explainable artificial intelligence \(XAI\) in financial time series forecasting](#). *ACM Comput. Surv.*, 57(10):265:1–265:37.
- David S Broomhead and Roger Jones. 1989. Time-series analysis. *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences*, 423(1864):103–121.
- Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, and 1 others. 2025. [MLE-Bench: Evaluating machine learning agents on machine learning engineering](#). In *International Conference on Learning Representations*.
- Ching Chang, Jeehyun Hwang, Yidan Shi, Haixin Wang, Wei Wang, Wen-Chih Peng, and Tien-Fu Chen. 2026. Time-imm: A dataset and benchmark for irregular multimodal multivariate time series. *Advances in Neural Information Processing Systems*, 38.
- Ching Chang, Yidan Shi, Defu Cao, Wei Yang, Jeehyun Hwang, Haixin Wang, Jiacheng Pang, Wei Wang, Yan Liu, Wen-Chih Peng, and Tien-Fu Chen. 2025a. [A survey of reasoning and agentic systems in time series with large language models](#). *CoRR*, abs/2509.11575.
- Ching Chang, Yidan Shi, Defu Cao, Wei Yang, Jeehyun Hwang, Haixin Wang, Jiacheng Pang, Wei Wang, Yan Liu, Wen-Chih Peng, and 1 others. 2025b. A survey of reasoning and agentic systems in time series with large language models. *arXiv preprint arXiv:2509.11575*.
- Chao Chen, Karl Petty, Alexander Skabardonis, Pravin Varaiya, and Zhanfeng Jia. 2001. Freeway performance measurement system: mining loop detector data. *Transportation research record*, 1748(1):96–102.
- Chao Chen, Tian Zhou, Yanjun Zhao, Hui Liu, Rong Jin, and Liang Sun. 2025a. [Does vector quantization fail in spatio-temporal forecasting? exploring a differentiable sparse soft-vector quantization approach](#). In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, KDD ’25, page 143–154, New York, NY, USA. Association for Computing Machinery.
- Jialin Chen, Aosong Feng, Ziyu Zhao, Juan Garza, Gaukhar Nurbek, Cheng Qin, Ali Maatouk, Leandros Tassioulas, Yifeng Gao, and Rex Ying. 2025b. Mtbench: A multimodal time series benchmark for temporal reasoning and question answering. *arXiv preprint arXiv:2503.16858*.
- Edward Choi, Mohammad Taha Bahadori, Jimeng Sun, Joshua Kulas, Andy Schuetz, and Walter Stewart. 2016. [Retain: An interpretable predictive model for healthcare using reverse time attention mechanism](#). In *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc.
- Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. 2024. [A decoder-only foundation model for time-series forecasting](#). In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 10148–10167. PMLR.
- Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, and 1 others. 2025. A comprehensive survey of self-evolving ai agents: A new paradigm bridging foundation models and lifelong agentic systems. *arXiv preprint arXiv:2508.07407*.
- Tairan Fu, Raquel Ferrando, Javier Conde, Carlos Arriaga, and Pedro Reviriego. 2024. Why do large language models (llms) struggle to count letters? *arXiv preprint arXiv:2412.18626*.

- Juan L Gamella, Jonas Peters, and Peter Bühlmann. 2024. The causal chambers: Real physical systems as a testbed for ai methodology. *arXiv preprint arXiv:2404.11341*.
- Azul Garza, Cristian Challu, and Max Mergenthaler-Canseco. 2023. [TimeGPT-1](#). *arXiv preprint arXiv:2310.03589*.
- Rakshitha Godahewa, Christoph Bergmeir, Geoffrey I Webb, Rob J Hyndman, and Pablo Montero-Manso. 2021. Monash time series forecasting archive. *arXiv preprint arXiv:2105.06643*.
- Mononito Goswami, Konrad Szafer, Arjun Choudhry, Yifu Cai, Shuo Li, and Artur Dubrawski. 2024. [MO-MENT: A family of open time-series foundation models](#). In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 16115–16152. PMLR.
- Xiangming Gu, Tianyu Pang, Chao Du, Qian Liu, Fengzhuo Zhang, Cunxiao Du, Ye Wang, and Min Lin. 2025. When attention sink emerges in language models: An empirical view. In *International Conference on Learning Representations*, volume 2025, pages 97114–97144.
- Siyuan Guo, Cheng Deng, Ying Wen, Hechang Chen, Yi Chang, and Jun Wang. 2024a. [DS-Agent: Automated data science by empowering large language models with case-based reasoning](#). In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 16813–16848. PMLR.
- Yangyang Guo, Yanjun Zhao, Sizhe Dang, Tian Zhou, Liang Sun, and Yi Qian. 2024b. [Less is more: Embracing sparsity and interpolation with esiformer for time series forecasting](#). *Preprint*, arXiv:2410.05726.
- Zelin He, Sarah Alnegheimish, and Matthew Reimherr. 2026. Harnessing vision-language models for time series anomaly detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 21690–21698.
- Sirui Hong, Yizhang Lin, Bang Liu, Bangbang Liu, Binhao Wu, Ceyao Zhang, Danyang Li, Jiaqi Chen, Jiayi Zhang, Jinlin Wang, and 1 others. 2025. [Data interpreter: An LLM agent for data science](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 19796–19821.
- Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, and 1 others. 2025. Memory in the age of ai agents. *arXiv preprint arXiv:2512.13564*.
- Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. 2024. [MLAgentBench: Evaluating language agents on machine learning experimentation](#). In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 20271–20309. PMLR.
- Qihe Huang, Zhengyang Zhou, Yangze Li, Kuo Yang, Binwu Wang, and Yang Wang. 2026a. Many minds, one goal: Time series forecasting via sub-task specialization and inter-agent cooperation. *Advances in Neural Information Processing Systems*, 38:166475–166506.
- Wei-Chieh Huang, Weizhi Zhang, Yueqing Liang, Yuanchen Bei, Yankai Chen, Tao Feng, Xinyu Pan, Zhen Tan, Yu Wang, Tianxin Wei, and 1 others. 2026b. Rethinking memory mechanisms of foundation agents in the second half: A survey. *arXiv preprint arXiv:2602.06052*.
- Garima Jain and Bhawna Mallick. 2017. A study of time series models arima and ets. *Available at SSRN 2898968*.
- Daniel Jarrett, Jinsung Yoon, Ioana Bica, Zhaozhi Qian, Ari Ercole, and Mihaela van der Schaar. 2021. [Clairvoyance: A pipeline toolkit for medical time series](#). In *International Conference on Learning Representations*.
- Yushan Jiang, Kanghui Ning, Zijie Pan, Xuyang Shen, Jingchao Ni, Wenchao Yu, Anderson Schneider, Haifeng Chen, Yuriy Nevmyvaka, and Dongjin Song. 2025. [Multi-modal time series analysis: A tutorial and survey](#). In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, V.2, KDD 2025, Toronto ON, Canada, August 3-7, 2025*, pages 6043–6053. ACM.
- Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, and 1 others. 2024a. Time-llm: Time series forecasting by reprogramming large language models. In *International conference on learning representations*, volume 2024, pages 23857–23880.
- Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y. Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, and Qingsong Wen. 2024b. [Time-llm: Time series forecasting by reprogramming large language models](#). In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.
- Ming Jin, YiFan Zhang, Wei Chen, Kexin Zhang, Yuxuan Liang, Bin Yang, Jindong Wang, Shirui Pan, and Qingsong Wen. 2024c. [Position: What can large language models tell us about time series analysis](#). In *Forty-first International Conference on Machine Learning*.
- Baoyu Jing, Sanhorn Chen, Lecheng Zheng, Boyu Liu, Zihao Li, Jiaru Zou, Tianxin Wei, Zhining Liu, Zhichen Zeng, Ruizhong Qiu, and 1 others. 2026. Tsqaq: Time series analysis question and answering benchmark. *arXiv preprint arXiv:2601.23204*.
- Seil Kang, Jinyeong Kim, Junhyeok Kim, and Seong Jae Hwang. 2025. See what you are told: Visual attention sink in large multimodal models. *arXiv preprint arXiv:2503.03321*.

- Ryan King, Tianbao Yang, and Bobak J. Mortazavi. 2023. [Multimodal pretraining of medical time series and notes](#). In *Machine Learning for Health, ML4H@NeurIPS 2023, 10 December 2023, New Orleans, Louisiana, USA*, Proceedings of Machine Learning Research, pages 244–255. PMLR.
- Lingdong Kong, Xian Sun, Wei Chow, Linfeng Li, Kevin Qinghong Lin, Xuan Billy Zhang, Song Wang, Rong Li, Qing Wu, Wei Gao, and 1 others. 2026. Ai for auto-research: Roadmap & user guide. *arXiv preprint arXiv:2605.18661*.
- Yaxuan Kong, Yiyuan Yang, Yoontae Hwang, Wenjie Du, Stefan Zohren, Zhangyang Wang, Ming Jin, and Qingsong Wen. 2025a. Time-mqa: Time series multi-task question answering with context enhancement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 29736–29753.
- Yaxuan Kong, Yiyuan Yang, Shiyu Wang, Chenghao Liu, Yuxuan Liang, Ming Jin, Stefan Zohren, Dan Pei, Yan Liu, and Qingsong Wen. 2025b. [Position: Empowering time series reasoning with multimodal llms](#). *CoRR*, abs/2502.01477.
- Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. 2025. LLMs get lost in multi-turn conversation. *arXiv preprint arXiv:2505.06120*.
- Geon Lee, Wenchao Yu, Kijung Shin, Wei Cheng, and Haifeng Chen. 2025. Timecap: Learning to contextualize, augment, and predict time series events with large language model agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 18082–18090.
- Haoyang Li, Xuejia Chen, Zhanchao Xu, Darian Li, Nicole Hu, Fei Teng, Yiming Li, Luyu Qiu, Chen Jason Zhang, Li Qing, and 1 others. 2025a. Exposing numeracy gaps: A benchmark to evaluate fundamental numerical abilities in large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 20004–20026.
- Yangning Li, Weizhi Zhang, Yuyao Yang, Wei-Chieh Huang, Yaozu Wu, Junyu Luo, Yuanchen Bei, Henry Peng Zou, Xiao Luo, Yusheng Zhao, and 1 others. 2025b. A survey of rag-reasoning systems in large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 12120–12145.
- Yuante Li, Xu Yang, Xiao Yang, Xisen Wang, Weiqing Liu, and Jiang Bian. 2026a. R&d-agent-quant: a multi-agent framework for data-centric factors and model joint optimization. *Advances in Neural Information Processing Systems*, 38.
- Zhonghang Li, Lianghao Xia, Jiabin Tang, Yong Xu, Lei Shi, Long Xia, Dawei Yin, and Chao Huang. 2024. Urbangpt: Spatio-temporal large language models. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*, pages 5351–5362.
- Zihao Li, Xiao Lin, Zhining Liu, Jiaru Zou, Ziwei Wu, Lecheng Zheng, Dongqi Fu, Yada Zhu, Hendrik F. Hamann, Hanghang Tong, and Jingrui He. 2025c. [Language in the flow of time: Time-series-paired texts weaved into a unified temporal narrative](#). *CoRR*, abs/2502.08942.
- Zihao Li, Lecheng Zheng, Bowen Jin, Dongqi Fu, Baoyu Jing, Yikun Ban, Jingrui He, and Jiawei Han. 2025d. Can graph neural networks learn language with extremely weak text supervision? In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11138–11165.
- Zihao Li, Jiaru Zou, Feihao Fang, Xuying Ning, Mengting Ai, Tianxin Wei, Sirui Chen, Xiyuan Yang, and Jingrui He. 2026b. Heterogeneous scientific foundation model collaboration. *arXiv preprint arXiv:2604.27351*.
- Chenxi Liu, Shaowen Zhou, Qianxiong Xu, Hao Miao, Cheng Long, Ziyue Li, and Rui Zhao. 2025a. Towards cross-modality modeling for time series analytics: A survey in the llm era. *arXiv preprint arXiv:2505.02583*.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024a. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173.
- Penghang Liu, Elizabeth Fons, Svitlana Vyetrenko, Daniel Borrajo, Vamsi K. Potluru, and Manuela Veloso. 2025b. [Ts-agent: A time series reasoning agent with iterative statistical insight gathering](#). *CoRR*, abs/2510.07432.
- Xu Liu, Junfeng Hu, Yuan Li, Shizhe Diao, Yuxuan Liang, Bryan Hooi, and Roger Zimmermann. 2024b. [Unitime: A language-empowered unified model for cross-domain time series forecasting](#). In *Proceedings of the ACM on Web Conference 2024, WWW 2024, Singapore, May 13-17, 2024*, pages 4095–4106. ACM.
- Yong Liu, Haoran Zhang, Chenyu Li, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. 2024c. [Timer: Generative pre-trained transformers are large time series models](#). In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32369–32399. PMLR.
- Yubo Ma, Zhibin Gou, Junheng Hao, Ruochen Xu, Shuohang Wang, Liangming Pan, Yujiu Yang, Yixin Cao, and Aixin Sun. 2024. Sciagent: Tool-augmented language models for scientific reasoning. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pages 15701–15736.
- Qianyu Meng, Yanan Wang, Liyi Chen, Qimeng Wang, Chengqiang Lu, Wei Wu, Yan Gao, Yi Wu, and Yao Hu. 2026. Agent harness for large language model agents: A survey.

- Mike A Merrill, Mingtian Tan, Vinayak Gupta, Thomas Hartvigsen, and Tim Althoff. 2024. [Language models still struggle to zero-shot reason about time series](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 3512–3533, Miami, Florida, USA. Association for Computational Linguistics.
- Llama Meta. 2024. [The llama 3 herd of models](#). *CoRR*, abs/2407.21783.
- Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. [A time series is worth 64 words: Long-term forecasting with transformers](#). In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.
- Xuying Ning, Dongqi Fu, Tianxin Wei, Mengting Ai, Jiaru Zou, Ting-Wei Li, Hanghang Tong, Yada Zhu, Hendrik Hamann, and Jingrui He. 2026a. Mc-search: Evaluating and enhancing multimodal agentic search with structured long reasoning chains. *arXiv preprint arXiv:2603.00873*.
- Xuying Ning, Dongqi Fu, Tianxin Wei, Wujiang Xu, and Jingrui He. 2025. Graph4mm: Weaving multimodal learning with structural information. *arXiv preprint arXiv:2510.16990*.
- Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiaru Zou, Mengting Ai, Zhining Liu, Ting-Wei Li, and 1 others. 2026b. Code as agent harness. *arXiv preprint arXiv:2605.18747*.
- Kashif Rasul, Arjun Ashok, Andrew Robert Williams, Hena Ghonia, Rishika Bhagwatkar, Arian Khorasani, Mohammad Javad Darvishi Bayazi, George Adamopoulos, Roland Riachi, Nadhir Hassen, and 1 others. 2023a. Lag-llama: Towards foundation models for probabilistic time series forecasting. *arXiv preprint arXiv:2310.08278*.
- Kashif Rasul, Arjun Ashok, Andrew Robert Williams, Hena Ghonia, Rishika Bhagwatkar, Arian Khorasani, Mohammad Javad Darvishi Bayazi, George Adamopoulos, Roland Riachi, Nadhir Hassen, and 1 others. 2023b. [Lag-Llama: Towards foundation models for probabilistic time series forecasting](#). *arXiv preprint arXiv:2310.08278*.
- Matthew Renze and Erhan Guven. 2024. [Self-reflection in LLM agents: Effects on problem-solving performance](#). *CoRR*, abs/2405.06682.
- Eli Schwartz, Leshem Choshen, Joseph Shtok, Sivan Doveh, Leonid Karlinsky, and Assaf Arbelle. 2024. Numerologic: Number encoding for enhanced llms’ numerical reasoning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 206–212.
- Manajit Sengupta, Yu Xie, Anthony Lopez, Aron Habte, Galen Maclaurin, and James Shelby. 2018. The national solar radiation data base (nsrdb). *Renewable and sustainable energy reviews*, 89:51–60.
- Oleksandr Shchur, Ali Caner Turkmen, Nick Erickson, Huibin Shen, Alexander Shirkov, Tony Hu, and Bernie Wang. 2023. Autogluon-timeseries: Automl for probabilistic time series forecasting. In *International Conference on Automated Machine Learning*, pages 9–1. PMLR.
- Aaditya K Singh and DJ Strouse. 2024. Tokenization counts: the impact of tokenization on arithmetic in frontier llms. *arXiv preprint arXiv:2402.14903*.
- Chenxi Sun, Hongyan Li, Yaliang Li, and Shenda Hong. 2024. Test: Text prototype aligned embedding to activate llm’s ability for time series. In *International Conference on Learning Representations*, volume 2024, pages 37854–37881.
- Mingtian Tan, Mike A Merrill, Vinayak Gupta, Tim Althoff, and Thomas Hartvigsen. 2024. [Are language models actually useful for time series forecasting?](#) In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Katherine Tieu, Dongqi Fu, Zihao Li, Ross Maciejewski, and Jingrui He. 2025. Learnable spatial-temporal positional encoding for link prediction. *arXiv preprint arXiv:2506.08309*.
- U.S. Bureau of Labor Statistics. 2024. Unemployment rate [various locations]. URL <https://fred.stlouisfed.org/>.
- Ville de Montréal. 2020. Interventions des pompiers de Montréal. URL <https://www.donneesquebec.ca/recherche/dataset/vmtl-interventions-service-securite-incendie-montreal>.
- Chengsen Wang, Qi Qi, Jingyu Wang, Haifeng Sun, Zirui Zhuang, Jinming Wu, Lei Zhang, and Jianxin Liao. 2025. Chattime: A unified multimodal time series foundation model bridging numerical and textual data. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 12694–12702.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022a. [Chain-of-thought prompting elicits reasoning in large language models](#). In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and 1 others. 2022b. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.
- Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H Chi, and 1 others. 2025. Evo-memory: Benchmarking llm agent test-time learning with self-evolving memory. *arXiv preprint arXiv:2511.20857*.

- Andrew Robert Williams, Arjun Ashok, Étienne Marcotte, Valentina Zantedeschi, Jithendaraa Subramanian, Roland Riachi, James Requeima, Alexandre Lacoste, Irina Rish, Nicolas Chapados, and Alexandre Drouin. 2025. [Context is key: A benchmark for forecasting with essential textual information](#). In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, Proceedings of Machine Learning Research. PMLR / OpenReview.net.
- Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. 2024a. [Unified training of universal time series forecasting transformers](#). In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, Proceedings of Machine Learning Research, pages 53140–53164. PMLR / OpenReview.net.
- Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. 2024b. [Unified training of universal time series forecasting transformers](#). In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 53140–53164. PMLR.
- Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. 2023. [Timesnet: Temporal 2d-variation modeling for general time series analysis](#). In *The Eleventh International Conference on Learning Representations*.
- Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. 2021. [Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting](#). In *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 22419–22430.
- Xingjian Wu, Junkai Lu, Zhe Li, Xiangfei Qiu, Jilin Hu, Chenjuan Guo, Christian S. Jensen, and Bin Yang. 2026. [Timeart: Towards agentic time series reasoning via tool-augmentation](#). *CoRR*, abs/2601.13653.
- Yingqian Wu, Qiushi Wang, Zefei Long, Rong Ye, Zhongtian Lu, Xianyin Zhang, Bingxuan Li, Wei Chen, Liwen Zhang, and Zhongyu Wei. 2025. [Fin-team: A multi-agent collaborative intelligence system for comprehensive financial scenarios](#). In *Natural Language Processing and Chinese Computing - 14th National CCF Conference, NLPCC 2025, Urumqi, China, August 7-9, 2025, Proceedings, Part II*, Lecture Notes in Computer Science, pages 443–455. Springer.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024a. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, volume 2024, pages 21875–21895.
- Yijia Xiao, Edward Sun, Di Luo, and Wei Wang. 2024b. [Tradingagents: Multi-agents LLM financial trading framework](#). *CoRR*, abs/2412.20138.
- Nan Xu and Xuezhe Ma. 2025. Llm the genius paradox: A linguistic and math expert’s struggle with simple word-based counting problems. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3344–3370.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. [React: Synergizing reasoning and acting in language models](#). In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.
- Wen Ye, Jinbo Liu, Defu Cao, Wei Yang, and Yan Liu. 2025. [When LLM meets time series: Can llms perform multi-step time series reasoning and inference](#). *CoRR*, abs/2509.01822.
- Fangxu Yu, Xingang Guo, Lingzhi Yuan, Haoqiang Kang, Hongyu Zhao, Lianhui Qin, Furong Huang, Bin Hu, and Tianyi Zhou. 2026. [Tsrbench: A comprehensive multi-task multi-modal time series reasoning benchmark for generalist models](#). *CoRR*, abs/2601.18744.
- Yangyang Yu, Zhiyuan Yao, Haohang Li, Zhiyang Deng, Yuechen Jiang, Yupeng Cao, Zhi Chen, Jordan W. Suchow, Zhenyu Cui, Rong Liu, Zhaozhao Xu, Denghui Zhang, Koduvayur Subbalakshmi, Guojun Xiong, Yueru He, Jimin Huang, Dong Li, and Qianqian Xie. 2024. [Fincon: A synthesized LLM multi-agent system with conceptual verbal reinforcement for enhanced financial decision making](#). In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.
- Yurun Yuan and Tengyang Xie. 2025. [Reinforce LLM reasoning through multi-agent reflection](#). In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, Proceedings of Machine Learning Research. PMLR / OpenReview.net.
- Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. [Are transformers effective for time series forecasting?](#) In *Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023, Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2023, Washington, DC, USA, February 7-14, 2023*, pages 11121–11128. AAAI Press.
- Wenqi Zhang, Yongliang Shen, Zeqi Tan, Guiyang Hou, Weiming Lu, and Yueting Zhuang. 2023. [Data-Copilot: Bridging billions of data and humans with autonomous workflow](#). *arXiv preprint arXiv:2306.07209*.

- Haokun Zhao, Xiang Zhang, Jiaqi Wei, Yiwei Xu, Yuting He, Siqi Sun, and Chenyu You. 2025a. [Time-seriesscientist: A general-purpose AI agent for time series analysis](#). *CoRR*, abs/2510.01538.
- YanJun Zhao, Ziqing Ma, Tian Zhou, Mengni Ye, Liang Sun, and Yi Qian. 2023. [Gcformer: An efficient solution for accurate and scalable long-term multivariate time series forecasting](#). In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, CIKM '23*, page 3464–3473, New York, NY, USA. Association for Computing Machinery.
- YanJun Zhao, Tianxin Wei, Jiaru Zou, Xuying Ning, Yuanchen Bei, Lingjie Chen, Simmi Rana, Wendy H. Yang, Hanghang Tong, and Jingrui He. 2026. [Papermind: Benchmarking agentic reasoning and critique over scientific papers in multimodal llms](#). *Preprint*, arXiv:2604.21304.
- YanJun Zhao, Tian Zhou, Chao Chen, Liang Sun, Yi Qian, and Rong Jin. 2025b. [Sparse-vq transformer: An ffn-free framework with vector quantization for enhanced time series](#). In *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5.
- Siru Zhong, Weilin Ruan, Ming Jin, Huan Li, Qingsong Wen, and Yuxuan Liang. 2025. [Time-vlm: Exploring multimodal vision-language models for augmented time series forecasting](#). *arXiv preprint arXiv:2502.04395*.
- Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, and 1 others. 2026. [Externalization in llm agents: A unified review of memory, skills, protocols and harness engineering](#). *arXiv preprint arXiv:2604.08224*.
- Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. [Informer: Beyond efficient transformer for long sequence time-series forecasting](#). In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 11106–11115.
- Tian Zhou, Peisong Niu, Liang Sun, Rong Jin, and 1 others. 2023. [One fits all: Power general time series analysis by pretrained lm](#). *Advances in neural information processing systems*, 36:43322–43355.
- Jiaru Zou, Yikun Ban, Zihao Li, Yunzhe Qi, Ruizhong Qiu, Ling Yang, and Jingrui He. 2026. [Transformer copilot: Learning from the mistake log in llm fine-tuning](#). *Advances in Neural Information Processing Systems*, 38:159927–159967.
- Jiaru Zou, Dongqi Fu, Sirui Chen, Xinrui He, Zihao Li, Yada Zhu, Jiawei Han, and Jingrui He. 2025. [Rag over tables: Hierarchical memory index, multi-stage retrieval, and benchmarking](#). *arXiv preprint arXiv:2504.01346*.

Appendix

Roadmap. In this appendix, we provide a detailed overview of our methodology and experimental setup. Appendix A motivates TIMECLAW by examining why directly trusting LLMs for contextualized time-series reasoning is unreliable. Appendix B details the fingerprint computation underlying the multimodal memory, including the context embedding and the series fingerprint construction. Appendix C describes the full memory construction and retrieval procedure, covering bank-level normalization and the two-stage retrieval mechanism. Appendix D provides extended related work on time-series foundation models, agentic reasoning and harness design, and LLM agents for data science. Appendix E reports experiment setup details. Appendix F presents the full experimental results, including component ablations across multiple backbone families, context-type and domain-wise breakdowns on CiK, memory-bank size sweeps on TSRBench, and fine-grained comparisons against open-source LLMs, VLMs, and TSLLMs. Appendix G describes the baselines and their references in detail. Appendix H provides qualitative case studies that illustrate the end-to-end behavior of TIMECLAW on representative contextualized time-series tasks. Finally, Appendix I provides our prompt templates.

The table of contents is provided below for quick navigation.

Table of Contents

A Why NOT Trusting LLMs for Contextualized Time Series?	16
B Fingerprint Computation for Multimodal Memory	19
B.1 Context Embedding	19
B.2 Series Fingerprint	19
C Full Memory Construction and Retrieval	21
C.1 Bank-Level Normalization	21
C.2 Two-Stage Retrieval	21
D Extended Related Work	22
E Experiment Setup Details	23
E.1 Dataset Statistics and Details	23
E.2 Supported Time Series Tools	25
E.3 Hyperparameters	26
E.4 Metrics	26
E.5 Implementation Details	27
F Full Experiment Results	27
G Baseline Details and References	28
H Case Study	31
I Prompt Templates	33

A Why NOT Trusting LLMs for Contextualized Time Series?

We provide a theoretical analysis of why directly trusting LLMs over serialized contextualized time series can be problematic. The analysis focuses on two sources of misalignment: numerical values are first mapped into a discrete token space, and long temporal evidence is then processed under a finite and unevenly allocated attention budget. We begin with the tokenization-induced mismatch between numerical distance and token distance.

Proposition 1 shows that numerical closeness is not necessarily preserved after tokenization. This provides a geometric explanation for why surface-token similarity may be a poor proxy for numerical similarity.

Proposition 1 (Token Distance Is Not Numerical Distance). *Let $\eta : \mathbb{R} \rightarrow \mathcal{V}^*$ be a tokenizer-induced representation from real values to token sequences. In general, edit distance or embedding distance between $\eta(x)$ and $\eta(x')$ is not order-preserving with respect to numerical distance $|x - x'|$.*

Proof. Tokenizers operate over character or subword patterns rather than numerical metric structure. Hence there can exist x_1, x_2, x_3 such that

$$|x_1 - x_2| < |x_1 - x_3|, \quad (13)$$

but

$$d_{\text{tok}}(\eta(x_1), \eta(x_2)) > d_{\text{tok}}(\eta(x_1), \eta(x_3)). \quad (14)$$

For example, under a character-level representation, $x_1 = 99.999999$, $x_2 = 100.0$, and $x_3 = 98.999999$ satisfy $|x_1 - x_2| = 0.000001 < 1.0 = |x_1 - x_3|$, while $\eta(99.999999)$ and $\eta(100.0)$ differ in more character positions than $\eta(99.999999)$ and $\eta(98.999999)$ (we can alter number of 9s until this holds). Therefore, token similarity does not generally preserve numerical proximity. $\square$

This mismatch is consistent with empirical findings that LLMs often struggle with numerical reasoning tasks such as arithmetic, numerical retrieval, and magnitude comparison, partly because numbers are represented through surface token patterns rather than continuous magnitudes (Li et al., 2025a). Prior work also shows that number-tokenization choices can substantially affect arithmetic performance, indicating that numerical reasoning is sensitive to the discrete token representation itself (Singh and Strouse, 2024). Similar token-level limitations appear in exact symbolic counting

tasks: recent studies report that LLMs can fail on simple letter-counting problems, where the model must reason over characters rather than semantic word meaning (Xu and Ma, 2025; Fu et al., 2024). The widely discussed “strawberry” counting example is an informal illustration of the same issue: a word may be represented as subword tokens rather than as explicit characters, so exact character-level counting is not directly aligned with the model’s token-level representation. This limitation is not unique to “strawberry”; for example, prior work reports that an LLM can answer that the word “confusable” contains two occurrences of the letter “a”, while the correct answer is one (Xu and Ma, 2025). Such examples suggest that even simple character-level operations may be unreliable when they are mediated by subword token representations rather than explicit symbolic access to characters.

Beyond pairwise distance distortion, tokenization can also obscure the internal structure of numbers. In decimal notation, each digit has a fixed place value, whereas subword tokenization may group different spans of digits into variable symbolic chunks. Proposition 2 formalizes this place-value mismatch.

Proposition 2 (Subword Tokenization Does Not Preserve Place-Value Structure). *Let $x \in \mathbb{N}$ be a decimal integer with digit expansion*

$$x = \sum_{k=0}^{m-1} d_k 10^k, \quad d_k \in \{0, \dots, 9\}. \quad (15)$$

Let $\eta(x) = (v_1, \dots, v_J) \in \mathcal{V}^$ be a tokenizer-induced subword representation of the decimal string of x . In general, there does not exist a fixed token-level decomposition*

$$x = \sum_{j=1}^J \phi(v_j) 10^{\alpha_j}, \quad (16)$$

where $\phi : \mathcal{V} \rightarrow \mathbb{Z}$ and α_j are tokenizer-independent place-value exponents determined only by token position j .

Proof. Decimal notation has a fixed digit-level positional structure: the contribution of digit d_k is always $d_k 10^k$. Subword tokenization, however, partitions the decimal string into variable-length segments. That is, a number may be tokenized as chunks of different lengths depending on surface string patterns, vocabulary entries, and tokenizer merge rules.

Consider two classical numerical constants whose decimal strings may be tokenized with different segment lengths, e.g.,

$$\begin{aligned}\eta(3.14159) &= (3, .14, 159) \\ \eta(2.71828) &= (2.7, 18, 28)\end{aligned}\quad (17)$$

where tokens are written by their corresponding string pieces for illustration. While 3.14159 and 2.71828 are familiar approximations of π and e , their token-level segmentations may expose different symbolic structures. In the first case, the token “3” covers the integer part, “.14” covers the decimal point together with the 10^{-1} and 10^{-2} places, and “159” covers the 10^{-3} , 10^{-4} , and 10^{-5} places. In the second case, the token “2.7” covers the integer part, the decimal point, and the 10^{-1} place, while “18” and “28” cover the 10^{-2} – 10^{-3} and 10^{-4} – 10^{-5} places, respectively. Therefore, the numerical place value associated with a token cannot be determined solely from its token index j .

Hence, unless the tokenizer is explicitly constrained to respect digit-level place values, the token sequence $\eta(x)$ does not provide a fixed token-level basis aligned with the decimal expansion of x . Numerical magnitude must therefore be reconstructed indirectly from variable-length symbolic chunks, rather than being natively represented by the tokenization. $\square$

The previous two propositions characterize representation-level mismatches: tokenization does not preserve either numerical geometry or decimal place-value structure. We next show that this mismatch propagates to the learning objective itself: predicting the next token in symbolic space is not equivalent to predicting the next value in numerical space.

Proposition 3 (Next-Token Prediction Is Not Next-Value Prediction). *Let $(\mathcal{V}^*, d_{\text{tok}})$ be the token-sequence space induced by a tokenizer $\eta : \mathbb{R} \rightarrow \mathcal{V}^*$, and let $(\mathbb{R}, d_{\text{num}})$ be the numerical space with $d_{\text{num}}(x, x') = |x - x'|$. Consider a temporal process $\{X_t\}_{t \geq 1}$ with $X_t \in \mathbb{R}$ and its tokenized observation $Z_t = \eta(X_t)$. A next-token predictor minimizes*

$$\mathcal{L}_{\text{tok}}(p) = \mathbb{E}[-\log p(Z_{t+1} \mid Z_{1:t})], \quad (18)$$

whereas a next-value predictor minimizes

$$\mathcal{L}_{\text{num}}(g) = \mathbb{E}[d_{\text{num}}(g(X_{1:t}), X_{t+1})^2]. \quad (19)$$

In general, an optimizer of $\mathcal{L}_{\text{tok}}$ is not an optimizer of $\mathcal{L}_{\text{num}}$.

Proof. The next-token objective is a proper scoring rule over the discrete symbolic space $\mathcal{V}^*$; its Bayes-optimal solution is the conditional token distribution

$$p^*(z \mid Z_{1:t}) = \mathbb{P}(Z_{t+1} = z \mid Z_{1:t}). \quad (20)$$

By contrast, under squared numerical loss, the Bayes-optimal next-value predictor is

$$g^*(X_{1:t}) = \mathbb{E}[X_{t+1} \mid X_{1:t}]. \quad (21)$$

These two Bayes solutions live in different spaces: p^* is a distribution over token strings, while g^* is an element of the numerical space $\mathbb{R}$.

For the two objectives to be equivalent, the tokenizer-induced representation would need to preserve the task-relevant numerical structure. In particular, there must exist a decoding map $\delta : \mathcal{V}^* \rightarrow \mathbb{R}$ such that token-level optimality implies numerical optimality:

$$\delta \left(\arg \max_{z \in \mathcal{V}^*} p^*(z \mid Z_{1:t}) \right) = \mathbb{E}[X_{t+1} \mid X_{1:t}] \quad (22)$$

for all conditional distributions of X_{t+1} . This condition does not hold in general. First, Proposition 1 shows that token distance is not order-preserving with respect to numerical distance. Therefore, errors that are small in token space need not be small in numerical space, and vice versa. Second, the token objective assigns loss according to symbolic likelihood rather than numerical deviation. For example, predicting the tokenization of 100.0 instead of 99.999999 may incur a large token-level discrepancy despite a small numerical error, while predicting a token string closer in surface form can be numerically farther away.

Hence, minimizing next-token prediction risk does not generally minimize next-value prediction risk. The two coincide only under additional assumptions, such as a value-sufficient tokenizer and a decoding rule that preserves the numerical geometry relevant to the task loss. $\square$

So far, the analysis concerns the numerical representation induced by tokenization. Moreover, contextualized time series introduce another challenge: temporal signals can be long, sparse, and distributed across many timestamps. Even when all serialized tokens fit into the nominal context window, relevant temporal evidence must compete with many other tokens under a fixed attention budget.

Proposition 4 (Long-Context Temporal Information Dilution). *Let a contextualized time series $\mathcal{X} = (\mathbf{X}_{1:T}, \mathcal{C})$ be serialized into a token sequence*

$$\mathbf{z}_{1:L} = \text{Tokenize}(\sigma(\mathcal{X})). \quad (23)$$

Assume that a temporal task depends on a set of relevant temporal positions $\mathcal{R} \subseteq [L]$. In a self-attention layer, the output at query position i is

$$\mathbf{h}_i = \sum_{j=1}^L \alpha_{ij} \mathbf{v}_j, \quad \sum_{j=1}^L \alpha_{ij} = 1, \quad \alpha_{ij} \geq 0. \quad (24)$$

If the attention mass assigned to the relevant positions is bounded by

$$A_i(\mathcal{R}) = \sum_{j \in \mathcal{R}} \alpha_{ij} \leq \epsilon, \quad (25)$$

then the contribution of task-relevant temporal evidence to $\mathbf{h}_i$ is at most ϵ in attention mass. Therefore, as L grows, any mechanism that spreads attention over many irrelevant tokens or concentrates attention on position-biased sink tokens can dilute the effective use of relevant temporal information.

Proof. Decompose the attention output into relevant and irrelevant parts:

$$\mathbf{h}_i = \sum_{j \in \mathcal{R}} \alpha_{ij} \mathbf{v}_j + \sum_{j \notin \mathcal{R}} \alpha_{ij} \mathbf{v}_j. \quad (26)$$

The total coefficient assigned to the relevant component is exactly

$$A_i(\mathcal{R}) = \sum_{j \in \mathcal{R}} \alpha_{ij}. \quad (27)$$

If $A_i(\mathcal{R}) \leq \epsilon$, then the relevant component contributes no more than an ϵ fraction of the convex combination defining $\mathbf{h}_i$. In particular, if $\|\mathbf{v}_j\| \leq M$ for all j , then

$$\left\| \sum_{j \in \mathcal{R}} \alpha_{ij} \mathbf{v}_j \right\| \leq \sum_{j \in \mathcal{R}} \alpha_{ij} \|\mathbf{v}_j\| \leq \epsilon M. \quad (28)$$

Thus, when relevant temporal positions receive little attention mass, their effect on the attention output is bounded regardless of their task relevance. Long serialized time series increase the number of irrelevant or weakly relevant tokens competing for the same unit attention mass. Moreover, if some tokens act as attention sinks and absorb disproportionate mass, the remaining mass available to temporally relevant positions is further reduced. This proves the claim. $\square$

This proposition shows that serialized temporal evidence competes within a fixed attention budget. In long time series, relevant timestamps may be sparse, located in the middle of the context, or separated by long lags. Empirically, LLMs often show degraded retrieval when relevant information is placed in the middle of a long context, a phenomenon known as “lost in the middle” (Liu et al., 2024a; Laban et al., 2025). Meanwhile, attention-sink studies show that some tokens can attract disproportionate attention mass independent of semantic relevance (Xiao et al., 2024a; Gu et al., 2025; Kang et al., 2025). Together, these effects suggest that even when a serialized time series fits within the nominal context window, its temporally relevant information may not be uniformly or reliably used.

Corollary 5 (No Guaranteed Length Extrapolation). *Suppose an LLM is trained on serialized sequences of length at most $L_{\max}$. For a test-time time series whose serialization length satisfies $L > L_{\max}$, performance on temporal dependencies requiring positions beyond the training length is not guaranteed by the training objective alone.*

Proof. The next-token training objective constrains the model on the support of the training distribution, whose sequence lengths are at most $L_{\max}$. For serialized sequences with $L > L_{\max}$, the model is evaluated outside this length support. Unless the architecture, positional encoding, or training procedure imposes an extrapolation-consistent structure, there can exist multiple extensions of the learned conditional distribution that agree on all training lengths but behave differently beyond $L_{\max}$. Therefore, training loss on lengths up to $L_{\max}$ alone does not identify a unique or guaranteed behavior for long serialized time series. $\square$

Together, these results suggest that text-only LLM workflows face both representation-level and process-level limitations for contextualized time series. Tokenization may distort numerical geometry and place-value structure; next-token prediction optimizes symbolic likelihood rather than numerical forecasting or decision risk; and long serialized contexts may dilute or miss temporally relevant evidence. These limitations motivate time-series-native agentic workflows that operate on temporal objects through structured analysis rather than relying solely on serialized text.

B Fingerprint Computation for Multimodal Memory

This section provides details of the fingerprint computation of the multimodal memory in TIMECLAW.

B.1 Context Embedding

Textual descriptor c_τ . For each task $\tau = (q, \mathcal{X}, \mathcal{Y}, y^*, \ell)$ we form a single textual descriptor c_τ by concatenating the framing fields available in $\mathcal{C}$ that identify *what the task is asking*, rather than the raw numerical signal. The concrete field set depends on the benchmark schema. For Context-is-Key (CiK), c_τ joins the background, scenario, and constraints fields in order. For TSRBench, c_τ joins the subtask identifier, domain, task name, the list of channel names, and the question text. For TSAIA, c_τ joins the question_type, the prompt, the inline data description, and either the per-task output constraint (for analysis items) or the option list (for multiple-choice items). Empty fields are skipped, and the resulting string is truncated to a fixed character budget (we use 8000 characters, well below the encoder’s 8192-token limit) so that pathological cases such as stringified million-element arrays cannot overflow the encoder.

Encoder. The descriptor is embedded by a frozen sentence-level encoder,

$$\phi = \Phi(c_\tau) \in \mathbb{R}^{d_\phi}. \quad (29)$$

We use OpenAI’s text-embedding-3-small, which produces L_2 -normalized vectors with $d_\phi = 1536$. Because the outputs are unit-norm, cosine similarity reduces to a dot product, $\cos(\phi_q, \phi_m) = \phi_q^\top \phi_m$, which we exploit at retrieval time. If the encoder rejects an input as too long, the descriptor is halved and re-submitted, with at most two rounds of halving before the call is treated as a hard failure.

Storage. At bank-building time, the embedding is computed once per record and stored alongside the trajectory; the bank therefore supports retrieval without re-embedding the entire history at query time. Records logged before the embedding mechanism was introduced carry a sentinel no-embedding mask and are simply skipped during the text retrieval stage.

B.2 Series Fingerprint

Setup. Let $\mathbf{X}_{1:T} \in \mathbb{R}^{N \times T}$ be the numerical signal of the task and let $\mathbf{X}^{(i)} \in \mathbb{R}^T$ denote its i -th

channel. After removing non-finite entries (NaN, $\pm\infty$) we write the finite values of channel i as $x_1^{(i)}, \dots, x_{n_i}^{(i)}$, where $n_i \leq T$. Channels with $n_i = 0$ are dropped, and we let N' be the number of remaining channels. A small constant $\epsilon = 10^{-12}$ is added to denominators where indicated to keep computations finite.

A per-channel feature is set to 0 in two degenerate cases: when $n_i < 3$ (too short for the statistic to be meaningful) and when the channel standard deviation σ_i vanishes (the channel is constant). These conventions guarantee that the aggregated fingerprint is always finite, never NaN.

Per-channel statistical features. For each channel with $n_i \geq 3$ and $\sigma_i > 0$,

$$\mu_i = \frac{1}{n_i} \sum_t x_t^{(i)}, \quad (30)$$

$$\sigma_i = \sqrt{\frac{1}{n_i-1} \sum_t (x_t^{(i)} - \mu_i)^2}, \quad (31)$$

$$\text{IQR}_i = q_{0.75}(x^{(i)}) - q_{0.25}(x^{(i)}), \quad (32)$$

$$z_t^{(i)} = (x_t^{(i)} - \mu_i) / \sigma_i. \quad (33)$$

On the z -scored signal we read off scale-free higher moments,

$$\text{skew}_i = \frac{1}{n_i} \sum_t (z_t^{(i)})^3, \quad (34)$$

$$\text{kurt}_i = \frac{1}{n_i} \sum_t (z_t^{(i)})^4 - 3, \quad (35)$$

together with a robust outlier rate via the median absolute deviation (MAD),

$$\tilde{x}_i = \text{median}(x^{(i)}), \quad (36)$$

$$\text{MAD}_i = \text{median}(|x^{(i)} - \tilde{x}_i|) + \epsilon, \quad (37)$$

$$\text{out}_i = \frac{1}{n_i} |\{t : |x_t^{(i)} - \tilde{x}_i| > 3 \text{MAD}_i\}|. \quad (38)$$

Per-channel trend features. A linear trend is fit on a unit-rescaled time axis so that its slope is comparable across different-length channels. With $u_t = (t-1)/(n_i-1)$ for $t = 1, \dots, n_i$, let

$$(a_i, b_i) = \arg \min_{a,b} \sum_t (x_t^{(i)} - a u_t - b)^2. \quad (39)$$

We record the z -scored slope and the linear-fit R^2 ,

$$\text{slope}_i = a_i / (\sigma_i + \epsilon), \quad (40)$$

$$R_i^2 = 1 - \frac{\sum_t (x_t^{(i)} - a_i u_t - b_i)^2}{\sum_t (x_t^{(i)} - \mu_i)^2 + \epsilon}. \quad (41)$$

Table 4: The 20 features of the series fingerprint ψ , in assembly order. “struct” features describe the channel pool as a whole; “per-ch” features are computed per channel and aggregated by mean (the standard deviation is additionally log-scaled); “mv” is the single multivariate-coupling feature.

#	Symbol	Type	Description
1	logLen	struct	$\log_{10}(\text{median}_i n_i + 1)$; log-scaled median per-channel finite length.
2	logChan	struct	$\log_2(N' + 1)$; log-scaled number of non-empty channels.
3	miss	struct	Fraction of entries in $\mathbf{X}_{1:T}$ that are non-finite (NaN or $\pm\infty$).
4	irreg	struct	Indicator that timestamps are present and unevenly spaced ($\text{CV}(\Delta t) > 0.01$).
5	meanZ	–	Reserved slot; identically 0 in the current implementation.
6	stdLog	per-ch	$\log_{10}(\frac{1}{N'} \sum_i \sigma_i + 1)$; log-scaled mean per-channel standard deviation.
7	$\overline{\text{IQR}/\sigma}$	per-ch	Mean per-channel IQR-to-std ratio; a robust dispersion fingerprint.
8	$\overline{\text{skew}}$	per-ch	Mean per-channel skewness on the z -scored signal.
9	$\overline{\text{kurt}}$	per-ch	Mean per-channel excess kurtosis on the z -scored signal.
10	$\overline{\text{slope}}$	per-ch	Mean per-channel z -scored linear-fit slope on a unit-rescaled time axis.
11	$\overline{R^2}$	per-ch	Mean per-channel coefficient of determination of the linear-trend fit.
12	$\overline{\rho(1)}$	per-ch	Mean autocorrelation at lag 1.
13	$\overline{\rho(\ell_2)}$	per-ch	Mean autocorrelation at lag $\lfloor \sqrt{n_i} \rfloor$.
14	$\overline{\rho(\ell_3)}$	per-ch	Mean autocorrelation at lag $\lfloor n_i/4 \rfloor$.
15	$\overline{\text{fftFreq}}$	per-ch	Mean dominant non-DC frequency from the per-channel rFFT, in cycles per sample.
16	$\overline{\text{fftPFrac}}$	per-ch	Mean fraction of non-DC spectral power held by the dominant bin.
17	$\overline{\text{specEnt}}$	per-ch	Mean normalized Shannon entropy of the non-DC spectrum (in $[0, 1]$).
18	$\overline{\text{cp}}$	per-ch	Mean cumulative-sum change-point rate Z_i/n_i .
19	$\overline{ \rho }$	mv	Mean absolute Pearson correlation across distinct channel pairs (0 if univariate).
20	out	per-ch	Mean per-channel fraction of points beyond 3 MAD_i from the channel median.

Per-channel autocorrelation features. Let $c_t^{(i)} = x_t^{(i)} - \mu_i$. The sample autocorrelation at lag $\ell \in \{1, \dots, n_i - 1\}$ is

$$\rho^{(i)}(\ell) = \frac{\sum_{t=1}^{n_i-\ell} c_t^{(i)} c_{t+\ell}^{(i)}}{\sum_t (c_t^{(i)})^2 + \epsilon}. \quad (42)$$

We probe three diagnostically useful lags — a short lag, a square-root-scale lag, and a quarter-length lag,

$$\ell_1^{(i)} = 1, \quad \ell_2^{(i)} = \lfloor \sqrt{n_i} \rfloor, \quad \ell_3^{(i)} = \lfloor n_i/4 \rfloor, \quad (43)$$

and record the three values $\rho^{(i)}(\ell_k^{(i)})$ for $k = 1, 2, 3$.

Per-channel spectral features. For $n_i \geq 8$, we compute the real FFT of the centered signal,

$$\widehat{S}_k^{(i)} = |\text{rFFT}(c^{(i)})_k|^2, \quad k = 0, 1, \dots, \lfloor n_i/2 \rfloor, \quad (44)$$

with bin frequencies $f_k = k/n_i$ in cycles per sample. Let $k_i^* = \arg \max_{k \geq 1} \widehat{S}_k^{(i)}$ be the dominant non-DC bin. We record the dominant frequency and the fraction of non-DC power it carries,

$$\text{fftFreq}_i = f_{k_i^*}, \quad (45)$$

$$\text{fftPFrac}_i = \frac{\widehat{S}_{k_i^*}^{(i)}}{\sum_{k \geq 1} \widehat{S}_k^{(i)} + \epsilon}, \quad (46)$$

together with the normalized Shannon entropy of the non-DC spectrum,

$$p_k^{(i)} = \widehat{S}_k^{(i)} / \sum_{k' \geq 1} \widehat{S}_{k'}^{(i)}, \quad (47)$$

$$\text{specEnt}_i = -\frac{1}{\log_2 K_i} \sum_{k \geq 1} p_k^{(i)} \log_2 p_k^{(i)}, \quad (48)$$

where $K_i = \lfloor n_i/2 \rfloor$ is the number of non-DC bins. Normalizing by $\log_2 K_i$ keeps $\text{specEnt}_i \in [0, 1]$ across series of different length. When $n_i < 8$ all three spectral features are set to 0.

Per-channel change-point feature. Let $d_t^{(i)} = x_{t+1}^{(i)} - x_t^{(i)}$ for $t = 1, \dots, n_i - 1$, and define the centered cumulative-sum series

$$S_t^{(i)} = \sum_{s=1}^t (d_s^{(i)} - \bar{d}^{(i)}), \quad (49)$$

where $\bar{d}^{(i)}$ is the mean of the differences. Letting Z_i be the number of sign changes in $\text{sign}(S^{(i)})$, we set

$$\text{cp}_i = Z_i/n_i. \quad (50)$$

A stable trajectory produces a CUSUM that drifts smoothly and crosses zero rarely, whereas a regime-shifting trajectory produces many crossings; cp_i is a cheap proxy for piecewise level change.

Structural and multivariate features. The remaining features describe the series as a whole

rather than any single channel. The four structural features are

$$\log\text{Len} = \log_{10}(\text{median}_i n_i + 1), \quad (51)$$

$$\log\text{Chan} = \log_2(N' + 1), \quad (52)$$

$$\text{miss} = 1 - \frac{\sum_i n_i}{NT}, \quad (53)$$

$$\text{irreg} = \mathbb{1}\left[\frac{\sigma(\Delta\mathbf{t})}{\mu(\Delta\mathbf{t})} > 0.01\right], \quad (54)$$

where $\Delta\mathbf{t}$ is the sequence of inter-sample time differences when timestamps are available, and $\text{irreg} = 0$ otherwise. The single multivariate coupling feature is the mean absolute Pearson correlation across distinct channel pairs,

$$|\overline{\rho}| = \frac{2}{N''(N'' - 1)} \sum_{i < j} |\rho_{ij}|, \quad (55)$$

where N'' is the number of non-degenerate channels (truncated to a common length $n_{\min} = \min_i n_i$ and excluded if zero-variance), and ρ_{ij} is the Pearson correlation between channels i and j . When $N'' < 2$ or $n_{\min} < 3$, $|\overline{\rho}| = 0$.

Aggregation across channels. For per-channel features other than the standard deviation, we average across the N' valid channels, $\overline{\text{feature}} = \frac{1}{N'} \sum_i \text{feature}_i$. The standard deviation is averaged and then log-scaled to prevent a single outlier channel from dominating the metric,

$$\text{stdLog} = \log_{10}\left(\frac{1}{N'} \sum_i \sigma_i + 1\right). \quad (56)$$

Assembly. The fingerprint stacks the features in the fixed order listed in Table 4,

$$\psi = \Psi(\mathbf{X}_{1:T}) \in \mathbb{R}^{20}. \quad (57)$$

Slot 5 is reserved for a future level-aware feature and is identically zero in the current implementation; we keep the slot so that extensions do not invalidate existing banks.

Stability properties. Two properties of Ψ are worth noting. First, Ψ is invariant under permutations of the channel index: every aggregator above is symmetric in i , so a bank populated under one ordering of channels can still retrieve a query that uses a different ordering. Second, the use of robust statistics in the outlier-rate, IQR, and median-based features means that replacing $O(1)$ values per channel with outliers shifts ψ by $O(1/n_i)$ on those features rather than by an unbounded amount; the fingerprint thus changes slowly under local corruption of the input.

C Full Memory Construction and Retrieval

This appendix gives the complete memory mechanism summarized in Section 3.4: the construction of the context embedding ϕ , the explicit specification of the series fingerprint ψ , the bank-level normalization that turns the raw fingerprint into a retrieval-ready key, and the two-stage procedure that combines the two modalities at query time.

C.1 Bank-Level Normalization

Because the components of ψ live on very different natural scales — $\log\text{Len}$ is $O(1)$ but kurt can range over orders of magnitude — a naive L_2 distance would be dominated by whichever feature happens to have the largest variance in the bank. We therefore z -score each feature using the bank’s own per-feature mean and standard deviation. Let

$$\mu_\psi = \frac{1}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} \psi_m, \quad (58)$$

$$\sigma_\psi = \text{std}(\{\psi_m\}_{m \in \mathcal{M}}), \quad (59)$$

both computed coordinate-wise. Any coordinate with $\sigma_{\psi,d} < 10^{-9}$ is treated as constant in the bank and is excluded from the metric by setting $\sigma_{\psi,d} \leftarrow 1$; this collapses that coordinate to 0 post-normalization. The bank-normalized fingerprint is

$$\tilde{\psi} = (\psi - \mu_\psi) \oslash \sigma_\psi, \quad (60)$$

where $\oslash$ denotes coordinate-wise division. The scaler is recomputed lazily on every fresh load of the bank rather than persisted, so that bank growth is reflected immediately at the next query.

C.2 Two-Stage Retrieval

Given a query task τ_q with keys (ψ_q, ϕ_q) , retrieval proceeds in two stages that combine the two modalities.

Stage 1 (text-side candidate pool). We select an intermediate pool of size N by cosine similarity in the context-embedding space,

$$\mathcal{N}(\phi_q) = \text{Top-}N \phi_q^\top \phi_m, \quad (61)$$

where $\mathcal{M}^\phi \subseteq \mathcal{M}$ denotes the subset of records that carry a valid text embedding. The dot-product form is exact because both ϕ_q and the stored ϕ_m are unit-norm. In our experiments we use $N = 20$.

Stage 2 (signal-side ranking). Within the text-side candidate pool, we rank by L_2 distance in the bank-normalized fingerprint space,

$$\text{Retrieve}(\tau_q; k) = \text{Top-}k_{m \in \mathcal{N}(\phi_q)} - \|\tilde{\psi}_q - \tilde{\psi}_m\|_2. \quad (62)$$

k is a retrieval parameter that can be adjusted according to the LLM context window and memory density.

Optional filters. Two filters can be applied in either stage. A *family filter* restricts the candidate pool to records sharing the query’s task-family tag, which is useful for diagnostic settings that isolate the contribution of within-family retrieval. A *self-exclusion filter* removes the query’s own record, which matters when the bank and the query set overlap (e.g. during cross-validation). Both filters are off by default in our main experiments.

Complexity. Stage 1 costs $O(|\mathcal{M}| \cdot d_\phi)$ in time and is dominated by a single dense matrix-vector product. Stage 2 costs $O(N \cdot d_\psi)$. At our scale ($|\mathcal{M}|$ in the low thousands, $d_\phi = 1536$, $d_\psi = 20$, $N = 20$), the entire retrieval is sub-millisecond on a CPU and does not require an approximate-nearest-neighbor index.

D Extended Related Work

Time-Series Foundation Models. Recent time-series foundation models seek to reduce reliance on dataset-specific training through large-scale pre-training and transfer across heterogeneous temporal domains. TimeGPT demonstrates zero-shot forecasting across unseen series (Garza et al., 2023), while Lag-Llama targets probabilistic zero-shot and few-shot forecasting (Rasul et al., 2023b). Other recent work scales this paradigm along complementary axes: TimesFM uses a decoder-only patched forecasting architecture (Das et al., 2024), Chronos formulates forecasting as language modeling over quantized numerical tokens (Ansari et al., 2024b), Moirai studies universal forecasting across variables and frequencies (Woo et al., 2024b), MOMENT broadens pretraining toward general-purpose time-series representation learning (Goswami et al., 2024), and Timer studies generative pretraining for forecasting (Liu et al., 2024c). Most of the models uses transformer backbones yet other architectures are worth exploring (Ai et al., 2025b,a; Zou et al., 2026; Tieu et al., 2025). Together, these models offer reusable pre-trained components for forecasting and related

time-series tasks (Chen et al., 2025a; Guo et al., 2024b). They remain primarily model-centric, however: the interface is typically a predefined task such as forecasting, imputation, classification, or anomaly detection (Zhou et al., 2023; Zhao et al., 2023, 2025b). The interface is typically tied to predefined task formats. In contrast, TIMECLAW targets open-ended contextualized time-series tasks through an agentic pipeline, where the system can interpret task goals, reason over dataset context and temporal structure, and coordinate executable analyses beyond a fixed task formulation.

Agentic Reasoning and Harness. Recent advances in agentic systems extend LLMs from passive text generators to interactive problem solvers that operate through structured reasoning and execution harnesses (Li et al., 2025b; Ning et al., 2026a). An agent harness defines how an LLM policy interfaces with external computation and environments, including available tools, action schemas, state representations, observation formats, and execution constraints (Zhou et al., 2026). By externalizing specialized computation into this interface, harnesses reduce the burden on the LLM itself while making intermediate reasoning steps more controllable. Such harnesses are helpful to tool-augmented reasoning (Ma et al., 2024) and ReAct-style agents (Yao et al., 2023), where models decompose complex tasks into executable actions and ground intermediate reasoning in returned observations. Recent work further augments agent harnesses with memory modules for retrieving prior trajectories (Hu et al., 2025; Huang et al., 2026b) and evolving mechanisms that distill reusable skills or tools from past experience (Wei et al., 2025; Fang et al., 2025). Based on this harness paradigm, TIMECLAW specializes it for contextualized time-series reasoning through a time-series-native tool harness, episodic trajectory memory, and experience-driven tool evolution.

LLM Agents for Data Science. Language models have demonstrated strong capabilities in understanding structured data (Li et al., 2025d; Zou et al., 2025; Ning et al., 2025). LLM agents have also been developed for data analysis and machine-learning engineering, where agents write code, inspect intermediate outputs, and iterate over experiments (Kong et al., 2026; Li et al., 2026b; Zhao et al., 2026). Data-Copilot studies code-centric querying, processing, and visualization over large data sources (Zhang et al., 2023). MLAGentBench evaluates agents that modify files, run experiments,

Table 5: Domain-level composition of the CiK benchmark. CiK contains 71 context-aware temporal tasks covering seven real-world domains and a small set of synthetic dynamical-system tasks.

Domain / Source Type	# Tasks	Representative Data Source
Public Safety	26	Montreal fire department intervention logs (Ville de Montréal, 2020)
Climatology	12	Solar irradiance, cloud cover, and solar power production (Sengupta et al., 2018)
Transportation	11	PeMS traffic occupancy data (Chen et al., 2001)
Energy	7	Electricity consumption data (Godaheva et al., 2021)
Retail	6	ATM cash withdrawal data (Godaheva et al., 2021)
Mechanics	3	Causal Chambers wind-tunnel measurements (Gamella et al., 2024)
Economics	3	State- and county-level unemployment rates (U.S. Bureau of Labor Statistics, 2024)
Synthetic Dynamics	3	Simulated bivariate dynamical systems (Williams et al., 2025)

Table 6: Context-type composition of the CiK benchmark. Context-type tags are not mutually exclusive, so the counts do not sum to the total number of tasks.

Context Type	# Tasks	Meaning in CiK	Representative Example
Intemporal	51	Time-invariant information about the process, such as the nature of the target variable, long-period seasonal patterns, or value constraints.	A task may describe the temporal process and target variable, or state constraints such as positivity that are not fully inferable from the observed numerical history.
Historical	25	Information about past behavior that is not directly reflected in the available history window.	A public-safety task may provide aggregate past statistics, such as the yearly average number of incidents or the month with the highest incident count.
Covariate	52	Information about additional variables that are statistically associated with the target series.	In the Montreal fire examples, another incident type, such as field fires, may be provided as an associated variable that helps interpret seasonal behavior.
Future	33	Information about future events, scenarios, or constraints that are relevant to the future temporal behavior.	CiK includes scenarios such as an ATM being inaccessible during a future period, leading to zero withdrawals, or an incoming weather event that changes electricity demand.
Causal	14	Information about whether covariates causally affect, are caused by, or are confounded with the target variable.	In the Montreal fire examples, causal context is needed to decide whether a future intervention on one incident type should affect the target incident series.

and improve models across machine-learning tasks (Huang et al., 2024), while MLE-bench extends this setting to Kaggle-style machine-learning engineering competitions (Chan et al., 2025). Workflow-level agents such as DS-Agent and Data Interpreter further combine planning, tool use, execution feedback, and code refinement for end-to-end data-science workflows (Guo et al., 2024a; Hong et al., 2025). These systems are related to TIMECLAW in treating analysis as an interactive executable process rather than a fixed prediction interface. Their scope, however, is general data-science and ML-engineering automation, and time-series-specific evaluation are not the central design constraint. TIMECLAW focuses on this contextualized time-series analysis setting.

E Experiment Setup Details

E.1 Dataset Statistics and Details

E.1.1 Context-is-Key (CiK) Benchmark

We use the Context-is-Key (CiK) benchmark (Williams et al., 2025) as a testbed for evaluating whether models can incorporate essential natural-language context when reasoning over contextualized time series. CiK pairs numerical temporal observations with textual information that associates with the underlying real-world process. This design aligns with our setting where time series are not treated as isolated numerical sequences, but as partial observations embedded in broader contextual evidence. The provided context may include background knowledge about the process, future events, operational constraints, historical summaries, covariates, or

Table 7: Task taxonomy of TSRBench. The benchmark contains 15 fine-grained tasks grouped into four temporal reasoning dimensions. Domains are reported at a high level based on instance-level metadata and content inspection.

Dimension	# Tasks	Fine-Grained Tasks	Domains Covered	Main Capability Tested
Perception	4	Pattern Recognition; Noise Understanding; Anomaly Detection; Comparative Analysis	Various domains; patterns drawn from synthetic signals, signal processing, and statistical time series	Recognizing temporal patterns, noise, anomalies, and relationships among multiple series.
Reasoning	7	Etiological Reasoning; Causal Discovery; Abductive Reasoning; Temporal Relation Reasoning; Numerical Reasoning; Deductive Reasoning; Inductive Reasoning	Health; Hydrology; Physics; Political Science; Sports; Energy; Traffic; Agriculture; Astronomy; Chemistry; Meteorology; Seismology	Inferring causes, latent events, temporal order, quantitative relations, rules, and higher-level temporal principles.
Prediction	2	Time Series Forecasting; Event Prediction	Finance; Climate	Predicting future numerical values or future discrete events from temporal observations and contextual information.
Decision-Making	2	Qualitative Decision-Making; Quantitative Decision-Making	Finance; Health	Selecting decisions or actions by combining temporal patterns, contextual knowledge, rules, and quantitative outcome comparison.

Table 8: Subtask-level instance distribution in TSRBench and our evaluation split. We use a stratified 20% subset, selecting approximately one fifth of the instances from each subtask.

Dimension	Subtask	Full Instances	Our 20% Split
Perception	Pattern Recognition	371	74
Perception	Anomaly Detection	129	26
Perception	Similarity Analysis	113	23
Perception	Noise Understanding	87	17
Prediction	Event Prediction	360	72
Prediction	Time Series Forecasting	720	144
Decision-Making	Qualitative Decision-Making	335	67
Decision-Making	Quantitative Decision-Making	300	60
Reasoning	Abductive Reasoning	150	30
Reasoning	Causal Reasoning	300	60
Reasoning	Deductive Reasoning	250	50
Reasoning	Etiological Reasoning	350	70
Reasoning	Inductive Reasoning	100	20
Reasoning	Numerical Reasoning	400	80
Reasoning	Temporal Relation Reasoning	160	32
Total		4,125	825

causal relationships.

CiK contains 71 manually designed context-aware temporal tasks. Most tasks are based on real-world time series from various application domains, including climatology, economics, energy, mechanics, public safety, transportation, and retail. The benchmark draws from 2,644 publicly available time series, with sampling frequencies ranging from 10-minute intervals to monthly observations. A small fraction of the tasks use synthetic dynamical systems. Each task template can be instantiated with different time series, time windows, and language formulations, making CiK a useful benchmark for testing whether a model can ground

temporal reasoning in diverse contextual information.

Table 5 summarizes the task composition by domain. CiK also annotates each task with one or more context types, including intemporal information, historical information, covariate information, future information, and causal information. Table 6 summarizes these context types, their task counts, and representative examples from the benchmark. These context-type tags are not mutually exclusive: a single task may require multiple types of contextual information. We refer readers to the original CiK paper for full data-source descriptions, task-construction details, examples, and additional

visualizations (Williams et al., 2025).

E.1.2 TSRBench Benchmark

We use TSRBench (Yu et al., 2026) as a multi-task benchmark for evaluating contextualized temporal reasoning with generalist models. TSRBench is designed to assess whether models can interpret time series together with task instructions, contextual information, and domain-specific cues across diverse temporal reasoning scenarios. Unlike benchmarks centered on a single predefined temporal task, TSRBench organizes problems into multiple capability dimensions, making it well aligned with our setting of open-ended contextualized temporal workflows.

TSRBench contains 4,125 problem instances and 15,250 time-series channels spanning 14 domains. The benchmark covers four major reasoning dimensions: perception, reasoning, prediction, and decision-making. These dimensions are further divided into 15 fine-grained tasks, as summarized in Table 7. TSRBench also supports multiple representations of time series, including textual sequences, visualized plots, text-visual inputs, and time-series embeddings, enabling evaluation of different classes of generalist models.

Table 7 summarizes the high-level task taxonomy and domain coverage used in our TSRBench evaluation. The task taxonomy and domain coverage are not treated as a strict one-to-one hierarchy: a capability dimension may involve multiple domains, and domain coverage is summarized at a high level based on instance-level metadata and content inspection. We refer readers to the original TSRBench paper for detailed task construction procedures, data collection strategies, quality control, and example cases (Yu et al., 2026).

Due to budget constraints, we evaluate on a 20% subset of TSRBench, covering 825 out of 4,125 instances. This sampling is feasible because TSRBench contains repeated task templates; we sample balanced task coverage and verify in a small-scale study that results on the 20% subset closely match those on the full test set. The sampling is stratified by subtask, so each subtask contributes nearly one-fifth of its original instances to our evaluation split. Table 8 summarizes the full benchmark distribution and our evaluation subset.

E.1.3 TSAIA Finance Benchmark

We evaluate TIMECLAW on the TSAIA Finance benchmark (Ye et al., 2025), an applied financial benchmark designed around practical time-series

analysis scenarios. The benchmark requires models to interpret historical financial time series together with task-specific information such as portfolio weights, risk-free rates, and market-index references. These tasks reflect realistic analytical questions in financial decision-making, including portfolio risk comparison, risk-adjusted return assessment, and market-relative stock behavior analysis.

This benchmark further evaluates the practical applicability of TIMECLAW in financial time-series analysis, where temporal observations must be converted into domain-specific decisions. Table 9 summarizes the task composition used in our evaluation.

E.2 Supported Time Series Tools

Table 10 catalogs the default tools exposed by the harness $\mathcal{T}$. The eight seed tools (the inspection and forecasting families) are deliberately atomic, so that the policy retains full control over how they are composed during an analysis. The remaining three, marked with $\dagger$, were *not* pre-designed: they were autonomously distilled by TIMECLAW’s capability-evolution loop (Section 3.3) from the bank-building trajectories of prior runs.

Evolved Tools: a case study on time series capability-evolution. The three domain-specific tools, `portfolio_var`, `portfolio_sharpe`, and `capm_regression`, emerged after enough successful bank-building trajectories had accumulated on TSAIA, where each task asks the agent to compare option portfolios under a specific quantitative-finance metric. The tool-synthesis agent identified three recurring sub-procedures that the policy was repeatedly assembling from primitive inspection calls: (i) empirical or parametric tail estimation of weighted-portfolio log returns (VaR); (ii) annualized mean-to-volatility ratio of risk-free-adjusted log returns (Sharpe); and (iii) OLS regression of aligned asset and market log returns (CAPM). For each pattern, the synthesis agent proposed a self-contained tool with a typed signature, a docstring, and a body that operates on the workspace through the same interface as the seed tools. Each candidate was verified by held-out execution and admitted into $\mathcal{T}$ only because its success rate exceeded $\gamma = 0.7$. The mechanism is not finance-specific: the finance trio is the first concrete instance of harness self-extension, and analogous tools (e.g., ECG-style anomaly localization, climate-baseline subtraction) are expected to emerge as $\mathcal{M}$ accumulates trajectories from other domains.

Table 9: Task composition of the TSAIA Finance benchmark used in our experiments. The tasks cover portfolio risk, risk-adjusted return, and market-relative stock behavior.

Financial Task	# Instances	Task Description
VaR Confidence Level	50	Compare candidate portfolios under a value-at-risk criterion and identify the portfolio with the lowest downside risk over the target horizon.
Sharpe Ratio	50	Compare candidate portfolios using historical stock prices and risk-free-rate information to determine the strongest risk-adjusted return.
Market Alpha	25	Assess whether a stock outperforms or underperforms a market index after accounting for risk-adjusted market exposure.
Market Beta	25	Assess whether a stock is more or less sensitive to market movement by reasoning about its market-relative volatility.
Total	150	Applied financial temporal reasoning instances.

Table 10: Tools currently exposed in the executable harness ($\mathcal{T}$) of TIMECLAW, grouped by family. All tools operate on the task-local workspace $\mathcal{W}$ at full numerical precision and return compact structured results to the policy. Tools marked $\dagger$ were admitted into $\mathcal{T}$ via the capability-evolution loop of Section 3.3; the remainder are seed tools shipped with the harness.

Family	Tool	Description
Inspection	list_channels	Return the names of the channels currently loaded in $\mathcal{W}$.
	series_overview	Per-channel (n , min, max, mean) summary together with timestamp count and task metadata.
	channel_stats	Extended descriptive statistics on one channel: n , min, max, mean, std, median, q_{25} , q_{75} .
	channel_values	Bounded slice of raw values from one channel (≤ 500 elements per call); arguments start, end, stride.
	compute_acf	Sample autocorrelation from lag 0 to a user-specified max_lag.
Forecasting	detect_periodicity	FFT-based dominant period in samples and the fraction of non-DC spectral power it carries.
	find_peaks	Local maxima detected via <code>scipy.signal.find_peaks</code> ; returns indices, values, and count.
	arma_forecast	Fit ARIMA(p, d, q) on a channel and return a point forecast over the next periods steps, together with the in-sample AIC.
	portfolio_var †	Value-at-Risk of a weighted-price portfolio over a chosen horizon, via historical or parametric estimation.
	portfolio_sharpe †	Annualized Sharpe ratio of a weighted-price portfolio; the risk-free rate can be a constant or a per-period channel.
Domain-Specific (Finance)	capm_regression †	OLS regression of asset log returns on market log returns, returning α , β , and R^2 .

E.3 Hyperparameters

Table 11 lists the hyperparameters of TIMECLAW and the values used in our main experiments. The defaults were fixed before any aggregate result was inspected and are shared across CiK, TSRBench, and TSAIA-MC unless noted.

The subsample seed and the split seed are intentionally independent, so one can vary (e.g., for cross-validation across splits) without disturbing the other. The min-one-train-per-family policy ensures that every task family seen at test time has at least one same-family exemplar available in the bank. The bank-level standard-deviation floor catches fingerprint coordinates that are near-constant in early banks; affected coordinates col-

lapse to zero in the z -scored representation rather than triggering a division-by-zero. The reference renderer omits the precedent’s final answer for the anchoring reason discussed in Section 3.4. All experiments use 32 concurrent rollouts; worker count affects wall-clock time but not results.

E.4 Metrics

RCRPS Metric for Context-is-Key Benchmark.

We evaluate CiK with the official Region-of-Interest Continuous Ranked Probability Score (RCRPS) protocol (Williams et al., 2025). RCRPS is designed for context-aware temporal prediction, where the provided context may affect only a subset of the target time steps or impose constraints on valid trajectories. Compared with the

Table 11: Hyperparameters of TIMECLAW.

Category	Hyperparameter	Default Value
Backbone	LLM policy π_θ	gpt-5-nano
Train/test split	Train ratio	0.5
	Min train records per family	1
	Split seed	2026
	Subsample seed	42
Multimodal memory	Fingerprint dim d_ψ	20
	Encoder Φ	text-embedding-3-small
	Embedding dim d_ϕ	1536
	Context descriptor char cap	≤ 8000
	Bank std floor (z -score)	10^{-9}
Two-stage retrieval	Text-stage pool size N	20
	Top- k injected	3
Capability evolution	Verification threshold γ	0.7

standard CRPS, RCRPS introduces two context-specific components: a region of interest (RoI), which assigns higher importance to time steps where the context is most relevant, and a constraint-violation term, which penalizes trajectories that violate context-implied constraints.

For a CiK task instance τ , let $\hat{\mathcal{P}}_\tau$ denote the model’s predictive distribution over the target temporal trajectory $\mathbf{y}_\tau^*$. We write the CiK evaluation function as

$$\ell_{\text{CiK}}(\hat{\mathcal{P}}_\tau, \mathbf{y}_\tau^*) = \text{RCRPS}_\tau(\hat{\mathcal{P}}_\tau, \mathbf{y}_\tau^*). \quad (63)$$

Following the official CiK implementation, RCRPS combines the CRPS over the RoI, the CRPS over the remaining target steps, and a task-specific constraint-violation penalty:

$$\text{RCRPS}_\tau = \alpha_\tau \mathcal{E}_\tau + \mathcal{V}_\tau(\beta), \quad (64)$$

where $\mathcal{E}_\tau$ denotes the RoI-weighted predictive error, $\mathcal{V}_\tau(\beta)$ denotes the constraint-violation penalty, and α_τ is the task-specific scaling factor used by CiK to make scores comparable across tasks with different numerical scales. When an RoI is specified, CiK assigns equal total weight to RoI and non-RoI target steps; when no meaningful RoI is specified, the score is computed over the full target horizon. We use the official CiK evaluator without modification, including the official constraint-penalty setting $\beta = 10$, task-specific RoI sets, scaling factors, constraint functions, and aggregation protocol. Lower RCRPS indicates better performance.

sMAPE Metric for Context-is-Key Benchmark.

For deterministic trajectory evaluation on CiK, let $\mathbf{y}_\tau^*, \hat{\mathbf{y}}_\tau \in \mathbb{R}^H$ denote the gold and predicted continuations for a task instance τ over a horizon of

length H . We use the symmetric mean absolute percentage error (sMAPE) with a small denominator floor $\varepsilon = 10^{-2}$:

$$\text{sMAPE}_\tau = \frac{1}{H} \sum_{h=1}^H \frac{2|y_{\tau,h}^* - \hat{y}_{\tau,h}|}{\max(|y_{\tau,h}^*| + |\hat{y}_{\tau,h}|, \varepsilon)}. \quad (65)$$

Here $\text{sMAPE}_\tau \in [0, 2]$, and lower values indicate better deterministic trajectory accuracy.

E.5 Implementation Details

TIMECLAW is implemented in Python on top of LangChain¹ for the LLM-agent loop and the Model Context Protocol (MCP)² for the executable tool harness, with each task-local workspace $\mathcal{W}$ backed by an in-memory MCP server built using FastMCP³. Concurrent rollouts borrow isolated server-agent slots from a worker-indexed pool, so the workspace state of one task cannot leak into another. All experiments run on a Windows 11 machine with a 13th Gen Intel(R) Core(TM) i9-13900H CPU, 64GB RAM, and an NVIDIA RTX A4500 GPU; the code is platform-agnostic and runs unchanged on Linux. No local GPU is required, as the LLM policy is queried through the OpenAI API and all in-process computation is CPU-only. The Python version is 3.11.4; dependencies are listed in the README of the supplementary code.

F Full Experiment Results

In this section, we provide additional experimental results to complement the main paper. Figure 6

¹<https://www.langchain.com/>

²<https://modelcontextprotocol.io/>

³<https://gofastmcp.com/>

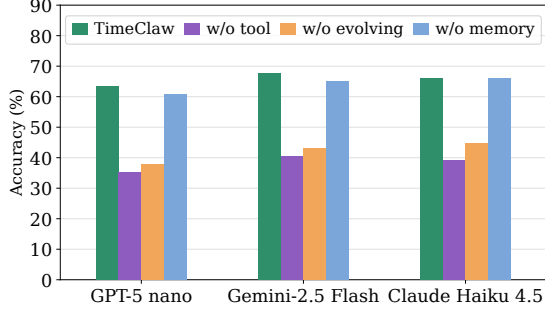


Figure 6: Across three backbone model families, removing tool use, capability evolution, or memory retrieval consistently degrades performance, validating the contribution of each component. The consistent gains of TIMECLAW across GPT, Gemini, and Claude backbones further show its compatibility with different model providers.

extends the component ablation on TSAIA to three backbone model families. Across GPT, Gemini, and Claude backbones, removing tool use, capability evolution, or memory retrieval consistently degrades performance. This confirms that each component contributes to TIMECLAW and further shows that the proposed agent harness is compatible with different model providers rather than being tied to a specific LLM family.

Figure 7 provides a context-type breakdown on the CiK benchmark. TIMECLAW achieves strong RCRPS across intemporal, historical, future, covariate, and causal information settings, indicating that the proposed framework can effectively leverage diverse forms of contextual information. We further report domain-wise CiK results in Table 12. TIMECLAW obtains the best average RCRPS, ranks first in five out of eight domains, and achieves the smallest average rank, demonstrating robust performance across heterogeneous real-world time-series domains. Figure 8 reports a memory-bank size ablation on TSRBench, sweeping the per-subtask training fraction, indicating that the multimodal memory bank is sample-efficient: a small training set already suffices to construct an effective bank, and TIMECLAW is robust to the size of the training corpus used. Finally, Table 13 provides fine-grained TSRBench results against open-source LLMs, VLMs, and TSLLMs. The results show that TIMECLAW achieves strong performance across perception, reasoning, prediction, and decision-making subtasks, outperforming substantially larger open-source models despite using a compact backbone.

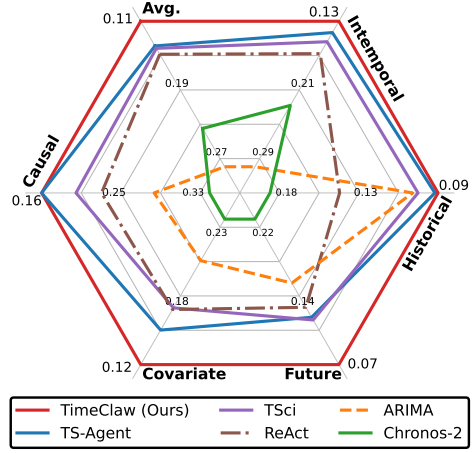


Figure 7: RCRPS comparison across CiK context types. Lower RCRPS is better; values closer to the outer boundary indicate stronger performance.

G Baseline Details and References

Due to the broad range of baselines considered in our experiments, we provide additional details in this appendix, including how each baseline is adapted to our evaluation settings when necessary. For complete methodological descriptions, we refer readers to the corresponding original publications.

Traditional Time Series Models. This group includes classical statistical models and neural models that operate directly on numerical time-series histories. These methods serve as non-agentic baselines for evaluating how far standard temporal modeling can go without explicit contextual reasoning or agentic workflow support.

- **ARIMA.** ARIMA is a classical statistical model that captures temporal dependence through autoregressive, differencing, and moving-average components.
- **ETS.** ETS uses exponential smoothing to model level, trend, and seasonal components. Like ARIMA, it relies on historical numerical patterns rather than natural-language context or agentic reasoning.
- **DLinear (Zeng et al., 2023).** DLinear decomposes a time series into trend and remainder components using a moving average, applies separate one-layer linear mappings to them, and sums the outputs for prediction.
- **PatchTST (Nie et al., 2023).** PatchTST is a Transformer-based model for long-term time-

Table 12: Domain-wise RCRPS results on the CiK benchmark. Lower values indicate better performance. The best and second-best results in each column are highlighted in **bold** and underlined, respectively. TIMECLAW achieves the best average RCRPS with a moderate token budget across all domains, ranks first in 5 out of 8 domains, and attains the smallest average rank of 2.625.

Method	Average RCRPS ($\downarrow$)	RCRPS by Domain ($\downarrow$)							
		Climatology	Economics	Energy	Mechanics	Public Safety	Retail	Synthetic	Traffic
<i>Traditional Time Series Models</i>									
ARIMA (1989)	0.2772	0.6717	0.3198	0.0964	0.7748	0.0938	0.2726	0.2817	0.2494
ETS (2017)	0.3282	0.8028	0.2837	0.0971	0.8036	0.1528	0.3054	0.3011	0.2740
DLinear (2023)	2.3448	1.9885	1.5496	0.1177	0.5057	5.0000	0.3214	0.3448	0.2425
PatchTST (2023)	2.3374	1.9903	1.3772	0.1107	0.5789	5.0000	0.2853	0.3319	0.2474
<i>Time Series Foundation Models</i>									
Chronos-2 (2025)	0.2344	0.1640	0.3439	0.0977	0.7142	0.1880	0.2990	0.3048	0.2925
Chronos-1-Large (2024a)	0.3362	0.3046	<u>0.2279</u>	0.0610	0.8840	0.1077	0.2178	0.3450	1.0284
Chronos-1-Tiny (2024a)	0.3380	0.2709	0.2931	0.0609	0.9166	0.1054	0.2397	0.3538	1.0411
Chronos-1-Small (2024a)	0.3474	0.2796	0.2427	0.0611	0.8412	0.1155	0.2248	0.3479	1.1126
Chronos-1-Mini (2024a)	0.3640	0.2837	0.2841	0.0610	0.9219	0.1105	0.2418	0.3513	1.1834
Lag-Llama (2023a)	0.2628	0.4049	0.4529	0.0615	0.8911	0.1190	0.2400	0.3414	0.3435
Moirai-Large (2024a)	0.4545	1.0599	0.2843	0.0613	0.7277	0.0986	0.2347	0.3233	1.0133
<i>LLM/Agent on Time Series</i>									
UniTime (2024b)	0.2822	0.6594	0.4928	0.0671	0.6988	<u>0.0931</u>	0.2794	0.3512	0.2660
Time-LLM (2024b)	0.3990	0.8478	0.5408	0.0678	0.8344	0.1163	0.3276	0.3625	0.6796
Time-LLM w/o context (2024b)	0.3912	0.7677	0.5938	0.0672	0.8014	0.1045	0.3205	0.3622	0.7438
Llama3-70B (2024)	0.2449	0.2958	0.2437	0.0611	0.8900	0.1034	0.1977	0.3583	0.4599
<i>General Agentic Pipelines</i>									
TS-Agent (2025b)	0.1421	0.1350	0.4037	0.0707	0.5449	0.0956	0.1747	0.1403	0.1070
TSci (2025a)	0.1448	<u>0.1056</u>	0.3845	0.0190	0.7685	0.1086	0.1643	<u>0.0878</u>	0.1224
CoT Prompt (2022a)	0.1726	0.1282	0.7298	0.0164	0.9519	0.1248	0.1332	0.0849	0.1142
ReAct (2023)	0.1514	0.1078	0.4218	<u>0.0144</u>	0.7077	0.1464	0.1396	0.1002	0.0929
Self-Reflection (2024)	0.1608	0.1129	0.3011	0.0195	1.1898	0.1255	<u>0.1379</u>	0.0946	0.0983
Multi-agent Refine (2025)	<u>0.1294</u>	0.1040	0.4221	0.0320	<u>0.5414</u>	0.1068	0.1456	0.1220	<u>0.0736</u>
TIMECLAW (Ours)	0.1145	0.1083	0.1571	0.0103	0.4203	0.0888	0.1713	0.1304	0.0730

series modeling. It segments each time series into subseries-level patches and treats these patches as input tokens to the Transformer. It also uses a channel-independent design, where different variables share the same embedding and Transformer weights.

Time Series Foundation Models. This group contains pretrained foundation models designed for broad time-series modeling across datasets and domains. These baselines test whether specialized temporal pretraining alone can match a harnessed agent that uses temporal tools, contextual reasoning, reusable routines, and memory.

- **Chronos-family models** (Ansari et al., 2024a, 2025). Chronos formulates time-series forecasting as language modeling over discrete value tokens: continuous observations are scaled, quantized into a fixed vocabulary, and modeled with Transformer language-model architectures. Through pretraining on diverse real and synthetic time series, Chronos-family models provide strong zero-shot probabilistic forecasting baselines.

- **Lag-Llama** (Rasul et al., 2023a). Lag-Llama is a pretrained foundation model for univariate probabilistic time-series forecasting. It uses a decoder-only Transformer architecture with lagged values as covariates and is pretrained on diverse time-series datasets to support zero-shot and fine-tuned prediction.

- **Moirai-family models** (Woo et al., 2024a). Moirai is a universal time-series forecasting Transformer trained for cross-domain zero-shot prediction. It is designed to handle heterogeneous temporal data, including different sampling frequencies, variable counts, and distributional properties, through a masked encoder architecture.

Language Models. This group includes general-purpose language models and time-series-oriented language-model baselines. These methods help evaluate whether language modeling or language-based adaptation alone is sufficient for contextualized temporal reasoning, without the full time-series-native harness used by TIMECLAW. In our experiments, we compared with a broader set of such general-purpose language models show im-

Table 13: Fine-grained TSRBench accuracy (%) of base models that are dominated by TimeClaw across all four task categories. Per-subtask numbers are taken from the original TSRBench paper (Yu et al., 2026). **Bold** marks the best score in each column; underline marks the second-best. By harnessing GPT-5-nano, TIMECLAW achieves the best performance, outperforming large open-source models with up to 235B parameters.

Model	Perception				Reasoning							Prediction		Decision		Overall
	PR	NU	AD	CA	ER	CD	AR	TR	NR	DR	IR	TSF	EP	QualDM	QuantDM	
Textual Time Series as Input																
Qwen2.5-3B	46.4	51.7	33.3	51.3	20.0	25.0	38.0	21.2	34.8	29.2	19.0	31.4	58.3	22.7	24.0	33.2
Qwen2.5-7B	50.7	50.6	41.1	54.9	15.1	32.3	46.0	28.1	33.2	24.0	28.0	33.5	37.5	31.0	25.7	33.7
Qwen2.5-72B	55.3	55.2	47.3	66.4	20.9	<u>58.0</u>	<u>62.0</u>	<u>33.8</u>	38.2	36.8	<u>40.0</u>	30.7	70.3	34.0	30.3	42.4
Qwen3-1.7B	45.8	59.8	38.0	51.3	18.3	30.1	48.7	30.0	27.0	28.4	20.0	39.2	67.8	31.3	24.7	36.8
Qwen3-8B	51.8	56.3	48.8	56.6	14.0	29.3	57.3	23.1	36.5	23.6	33.0	<u>46.7</u>	48.9	<u>34.9</u>	28.7	38.3
Qwen3-235B-A22B	66.0	56.3	<u>59.7</u>	67.3	22.6	34.7	86.7	28.1	44.8	49.2	39.0	29.0	48.9	34.8	30.0	42.2
Gemma3-12B-it	51.5	59.8	48.9	65.5	21.1	34.7	42.0	23.7	33.2	28.0	41.0	39.4	34.2	34.3	31.3	37.2
Gemma3-27B-it	56.1	59.8	48.1	68.1	22.3	37.0	<u>66.7</u>	21.9	36.0	29.3	35.0	36.1	42.8	33.1	30.3	38.6
Visual Time Series as Input																
Phi4-Multimodal-8B	52.3	46.0	41.9	48.7	24.6	22.3	28.7	23.1	30.5	25.2	26.0	32.1	25.8	<u>34.9</u>	29.0	31.9
InternVL3.5-1B	47.2	49.4	38.8	46.9	25.7	24.3	46.7	24.4	29.2	24.0	27.0	34.7	46.4	34.3	21.3	33.8
InternVL3.5-8B	60.9	55.2	52.7	64.6	25.7	38.3	60.0	27.5	40.5	31.6	29.0	38.2	41.9	33.4	22.3	39.5
InternVL3.5-38B	58.8	60.9	55.8	69.9	30.9	43.0	52.7	31.2	43.8	30.4	34.0	32.2	32.5	35.2	29.0	39.4
MiniCPM-V-4.5-8B	63.6	56.3	56.6	62.8	22.3	24.3	54.7	20.6	26.5	22.8	24.0	44.6	27.8	26.3	29.7	35.9
MiMo-VL-7B-RL	58.2	<u>64.4</u>	52.7	65.5	23.7	34.7	65.3	30.6	33.0	25.6	36.0	35.0	29.7	29.0	33.0	37.2
Embedded Time Series as Input																
OpenTSLM-3B	39.9	40.2	35.7	43.4	20.3	32.3	33.3	28.1	24.5	26.0	32.0	32.5	35.6	28.4	28.7	31.0
ChatTS-14B	50.7	50.6	46.5	55.8	21.7	34.3	51.3	24.4	30.5	23.6	25.0	19.2	52.2	29.3	27.7	33.5
TS-Reasoner-7B	53.1	56.3	48.1	50.4	28.9	24.3	57.3	26.9	37.2	23.6	24.0	31.7	55.0	32.8	21.7	36.4
TimeOmni-1-7B	55.0	59.8	41.9	64.6	28.0	35.3	46.7	24.4	31.5	22.8	34.0	30.0	49.4	34.0	30.3	36.7
TimeClaw (GPT-5-nano)	<u>76.9</u>	22.2	45.5	<u>72.7</u>	<u>45.7</u>	40.0	33.3	25.0	<u>72.5</u>	32.0	10.0	47.2	<u>83.3</u>	21.2	46.7	<u>49.8</u>
TimeClaw (GPT-5-mini)	84.6	66.7	81.8	81.8	48.6	76.7	53.3	50.0	80.0	<u>44.0</u>	<u>40.0</u>	38.9	86.1	21.2	<u>33.3</u>	57.3

proved performance over them. We provide details of representative models below. We refer the readers to the original benchmark papers (Williams et al., 2025; Yu et al., 2026) for more details.

- **LLaMA3-70B** (Meta, 2024). LLaMA3-70B is a strong open-weight general-purpose LLM from the Llama 3 family. For CiK, we use it as a text-only language-model baseline: the numerical time series and natural-language context are serialized into a single prompt, and the model generates the continuation autoregressively through next-token prediction. Under greedy decoding, each generated token is selected according to the model’s output logits.
- **UniTime** (Liu et al., 2024b). UniTime is a language-empowered unified model for cross-domain time-series forecasting. It uses domain instructions to provide domain-level information and a Language-TS Transformer to align language and time-series representations. The model aims to handle heterogeneous time series from different domains under a unified

architecture.

- **Time-LLM** (Jin et al., 2024b). Time-LLM reprograms pretrained LLMs for time-series forecasting while keeping the backbone language model frozen. It maps time-series patches into text-prototype representations aligned with the LLM input space, augments the input with Prompt-as-Prefix task information, and projects the LLM outputs into temporal predictions.

Agentic Frameworks. This group includes general prompting and agentic workflows that are not specifically designed for time series. They test whether generic LLM reasoning, tool-use, or reflection pipelines are sufficient for contextualized temporal tasks without specialized temporal run-time support.

- **Direct prompting.** Direct prompting feeds the serialized time series, context, and task instruction to the language model in a single prompt. The model directly produces the final answer without explicit reasoning steps, tool use, reflection, or multi-agent interaction.

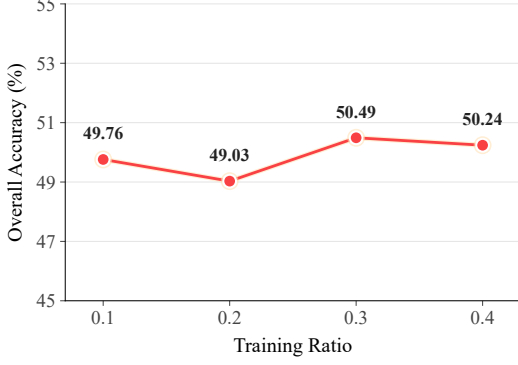


Figure 8: Overall accuracy on TSRBench as a function of the fraction of available trajectories used to construct the memory bank (with $k = 3$ retrieved neighbors at test time). Accuracy varies by less than 1.5% across $\text{train_ratio} \in \{0.1, 0.2, 0.3, 0.4\}$, indicating that the multimodal memory bank is sample-efficient: a small training set already yields a fully effective bank, and TIMECLAW is robust to the construction-set size.

- **Chain-of-thought prompting** (Wei et al., 2022a). Chain-of-thought prompting asks the language model to produce intermediate natural-language reasoning steps before the final answer. It is a prompting-only strategy for eliciting multi-step reasoning from large language models.
- **ReAct** (Yao et al., 2023). ReAct interleaves natural-language reasoning traces with task-specific actions. The reasoning steps help the model maintain and update its plan, while actions allow it to interact with external tools or environments. It is a general agentic prompting baseline for combining reasoning and tool use.
- **Self-reflection** (Renze and Guven, 2024). Self-reflection lets an LLM inspect its previous answer or reasoning process, identify possible errors, and use the generated feedback to revise its response. The authors study several reflection variants where agents learn guidance from prior mistakes before attempting the task again.
- **Multi-agent reflection** (Yuan and Xie, 2025). Multi-agent reflection uses multiple LLM agents to iteratively verify and improve answers. It models the refinement process as a Markov Decision Process and trains an actor-critic LLM system to incorporate feedback

over multiple rounds, improving reasoning through coordinated agent collaboration.

Time Series Agents. This group includes agentic methods specifically designed for time-series analysis. These baselines are closest to our setting, but they typically focus on particular temporal tasks or fixed agentic workflows rather than providing a general harness for contextualized temporal reasoning.

- **TS-Agent** (Liu et al., 2025b). TS-Agent is a time-series reasoning agent that lets LLMs synthesize conclusions from evidence while delegating statistical and structural extraction to tools over raw numerical sequences. It maintains an explicit evidence log and uses a self-critic with a final quality gate to iteratively refine reasoning.
- **TSci** (Zhao et al., 2025a). TSci is an LLM-driven agentic framework for general time-series forecasting. It organizes the workflow into specialized agents for data diagnostics, preprocessing, model planning, fitting, validation, ensembling, and reporting. The framework emphasizes automated forecasting together with transparent natural-language rationales and reports.

H Case Study

H.1 How TIMECLAW Leverages Context?

To test how TIMECLAW leverages textual context rather than pattern-matching on numerical history, we pair two SVAR tasks at the same seed: MINIMALCAUSALCONTEXTBIVARLINSVAR and FULLCAUSALCONTEXTEXPLICEQUATIONBIVARLINSVAR. The two records have byte-identical 128-step past history of X_1 , identical 32-step ground truth, and the same step-change schedule for the covariate X_0 ($0.028 \rightarrow 0.111 \rightarrow 0.223$). The Minimal prompt only lists the qualitative parents of X_1 ; the Full prompt additionally spells out the linear SVAR equation with all six coefficients. Any difference in forecast quality must therefore route through the agent’s reading of roughly 400 extra tokens of context.

Figure 11 overlays both forecasts on the shared ground truth. Given the equation, the agent inspects recent X_1 values through the analysis tools and unrolls the SVAR recursion on the known future X_0 schedule, recovering the ramp from 0.035

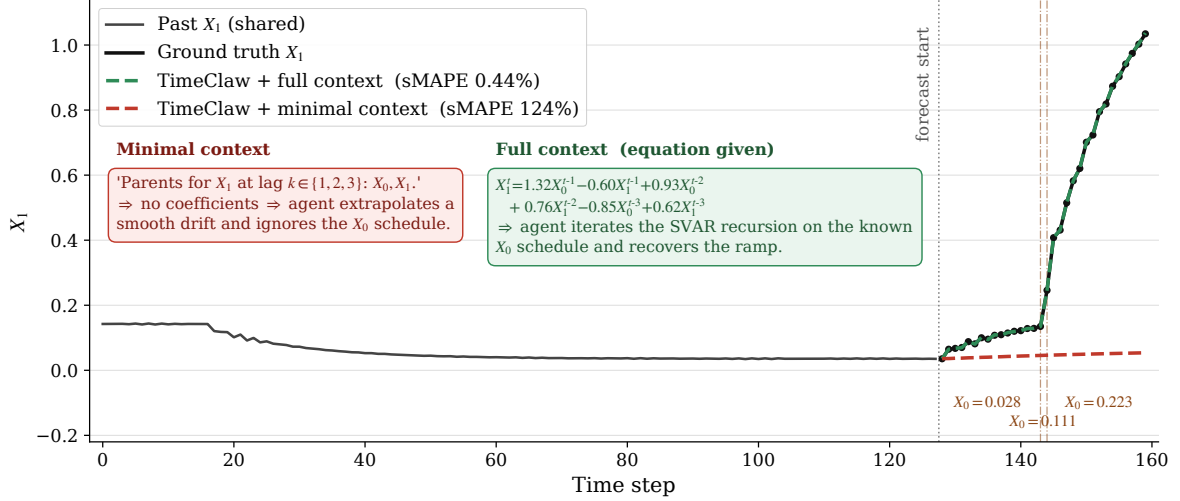


Figure 9: Case study on a paired SVAR task from Context-is-Key (Williams et al., 2025). Given identical history and ground truth, TIMECLAW recovers the ramp when the prompt contains the SVAR equation and misses it entirely when only the qualitative parent list is provided. This demonstrates that TIMECLAW actively grounds its forecast in the textual context, rather than extrapolating from the numerical history alone.

to 1.03 within $\sim 10^{-3}$ (sMAPE 0.44%). Without the coefficients it cannot translate the X_0 jumps into X_1 movements and falls back to a smooth drift inside $[0.036, 0.054]$, missing the ramp entirely (sMAPE 124%). The ~ 280 times gap on identical numerical inputs confirms that, when context carries decision-relevant structure, TIMECLAW propagates that structure into the forecast rather than relying on the history alone.

H.2 How Task Memory Steers Agent Decisions

To examine how retrieved task memory changes agent behavior, we trace TIMECLAW on a representative TSRBench causal-reasoning instance. The task asks the agent to infer the causal adjacency among multiple river sensors from the same underlying time series. We run the task under two settings with the same agent, MCP tool budget, and numerical input: retrieval disabled ($k=0$) and top-3 memory retrieval enabled ($k=3$). The retrieved memories come from the training bank and do not overlap with the test split.

Figure 10 contrasts the two outcomes. Without memory, the agent performs a broad numerical sweep, issuing 22 tool calls across `channel_stats`, `compute_acf`, `find_peaks`, and `detect_periodicity` for all channels. Despite this exhaustive inspection, it selects a physically reversed causal structure, mistaking downstream effects for upstream causes.

With memory, retrieval returns related causal-reasoning trajectories from sibling tasks. One retrieved memory summarizes a transferable rule: the relevant mechanism follows an upstream-to-downstream cascade. Guided by this rule, the agent avoids an exhaustive scan, runs only 7 targeted `compute_acf` probes to verify the lead-lag pattern, and selects the correct adjacency. Importantly, the retrieved memory provides no additional observations from the test instance; it transfers a compact decision rule learned from a different river system. Thus, memory changes both the answer and the reasoning process, reducing the tool budget by over $3\times$ while steering the agent toward the correct causal direction. A step-by-step trace is provided in Figure 12.

H.3 How TIMECLAW Evolves New Tools from Experience?

We further examine how TIMECLAW expands its toolset through experience-driven capability evolution. Tool evolution converts recurring successful trajectories into reusable executable routines. This mechanism is especially useful when the initial toolbox only provides generic time-series operations, while a benchmark repeatedly requires domain-specific computations that are cumbersome or unreliable to reconstruct from low-level tools. In such cases, TIMECLAW summarizes successful training trajectories, identifies repeated analytical routines, and promotes them into callable tools with

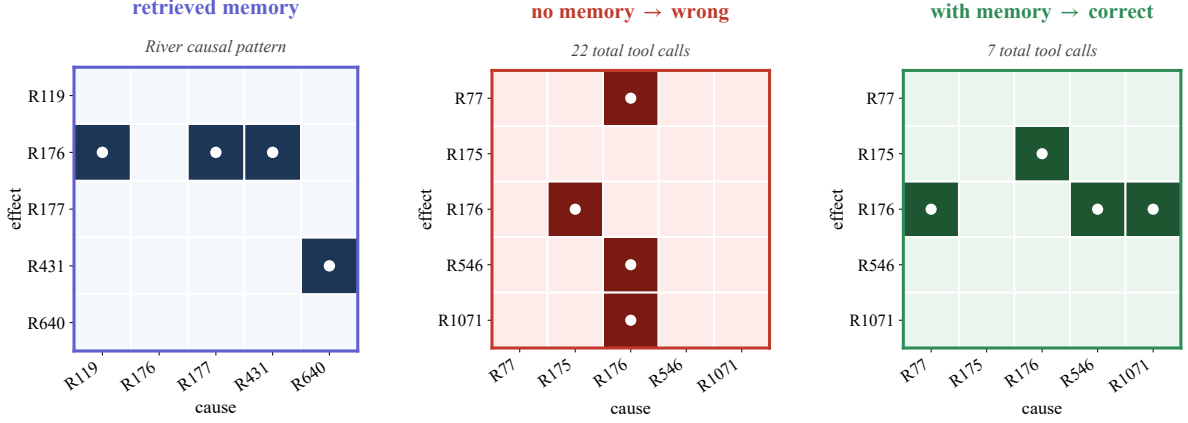


Figure 10: Causal maps of the case study on memory transfer in TSRBench river flood causal reasoning (R stands for "River"). **Left:** a retrieved memory summarizes an upstream-to-downstream cascade from a related but different river system, providing a transferable structural rule. **Middle:** without memory, the agent performs broad exploratory analysis and commits to a physically reversed adjacency after 22 tool calls. **Right:** with memory, the agent reuses the retrieved upstream-to-downstream reasoning pattern, executes only 7 targeted tool calls, and selects the correct causal structure. This illustrates that episodic memory improves reasoning by transferring compact task-level rules across related temporal scenarios, rather than by matching numerical histories alone.

standardized argument schemas. The evolved tools can then be reused at test time, allowing the agent to bypass brittle manual arithmetic and directly execute the intended computation.

Figure 13 illustrates this process on TSAIA’s finance split. Starting from a generic time-series toolbox, the agent observes recurring successful trajectories involving portfolio risk-adjusted return estimation, portfolio risk estimation, and market-factor regression. From these trajectories, TIMECLAW evolves three finance-specific tools: `portfolio_sharpe`, `portfolio_var`, and `capm_regression`. At test time, these evolved tools provide direct executable support for financial reasoning, improving both accuracy and tool-use efficiency. This case study shows that capability evolution transfers reusable computational procedures, rather than task-specific data, and enables the agent to acquire new domain-specific skills from prior experience.

I Prompt Templates

TIMECLAW instantiates a small family of prompts that share the same high-level skeleton but specialize the input rendering, the output schema, and the tool-use guidance to the underlying benchmark. All test-time prompts can optionally be wrapped with a retrieval prefix that injects top- k trajectory summaries from the episodic memory bank, and all training-time prompts are extended with a

ground-truth-aware suffix that elicits a transferable `<context_to_action>` explanation block. We describe the unified skeleton in Figure 14, and an example per-benchmark instantiations in Figure 15. We further describe the memory bank prompt in Figure 16.

Case Study C.1: TIMECLAW on Context-is-Key: How Textual Context Steers the Forecast

Core Task. We pick a pair of CiK tasks from the bivariate linear SVAR family at the same seed (seed=5): MINIMAL-CAUSALCONTEXTBIVARLINSVAR and FULLCAUSALCONTEXTEXPLICITEQUATIONBIVARLINSVAR. The two records share *identical* 128-step past history and 32-step ground-truth future of the target variable X_1 , and share the same step-change schedule for the covariate X_0 over the forecast window ($0.028 \rightarrow 0.111 \rightarrow 0.223$). The *only* difference is what the textual context reveals about the data-generating process. This isolates a clean variable: context $\rightarrow$ forecast.

Prompt: See Appendix I for the full prompt template.

Visualization: We provide Figure 9 to better illustrate this case study.

Memory Retrieval Stage.

1. **Two-stage retrieval.** TIMECLAW embeds the task’s background + scenario + constraints block with text-embedding-3-small, takes the top- K_{text} records by cosine similarity from the training bank, then re-ranks the survivors by L2 distance on the 20-dim numerical fingerprint of past_time. With $k = 1$ on this task both stages converge on the same family (FULLCAUSALCONTEXTIMPLICITEQUATIONBIVARLINSVAR) seen during training.
2. **Reference compression.** The retrieved trajectory is compressed by summarize_trajectory into the trainer’s analytic spine: a context $\rightarrow$ forecast explanation, together with the sequence of MCP tool calls and their truncated responses. This summary is then injected before the test prompt as a REFERENCES FROM PRIOR TRAINING block, surfacing the transferable rule that the scenario’s lagged linear parent effects should be handled by iterating the SVAR recursion over the known future X_0 schedule.

Execution Stage (Full Context).

1. **Context parsing.** The agent reads the explicit linear SVAR equation from the prompt ($X_1^t = 1.322 X_0^{t-1} - 0.604 X_1^{t-1} + 0.926 X_0^{t-2} + 0.763 X_1^{t-2} - 0.851 X_0^{t-3} + 0.623 X_1^{t-3}$) and the piece-wise constant schedule for X_0 over the 32-step horizon.
2. **Tool-grounded historical inspection.** Guided by the retrieved spine, the agent invokes the MCP analysis server: series_overview() $\rightarrow$ channel_stats("1") $\rightarrow$ compute_acf("1", max_lag=20) $\rightarrow$ channel_values("1", 120, 128), confirming the recent X_1 levels needed to seed the recursion.
3. **Closed-form forecast.** With the equation, the recent lagged values, and the future X_0 schedule all known, the agent unrolls the recursion deterministically, producing a 32-step forecast that tracks the ground-truth ramp from $0.035 \rightarrow 1.03$ almost exactly.

Counterfactual: Minimal Context, Same Series. On the matched MINIMAL-context variant, the prompt only states that “parents for X_1 at lag $k \in \{1, 2, 3\}$ are X_0, X_1 ,” while the six SVAR coefficients are withheld. Given the same retrieved memory and tool budget, the agent cannot reconstruct the recursion and instead falls back to extrapolating a small monotone drift in $[0.036, 0.054]$, missing the X_0 step changes that drive the true ramp.

Evaluation. On the full-context variant, TIMECLAW achieves **sMAPE 0.44%** ($\text{MSE} \approx 10^{-6}$), compared with **sMAPE 124%** ($\text{MSE} 0.254$) on the minimal-context variant. This yields a $\sim 280\times$ sMAPE gap for forecasting the *same* numerical series. Since the two variants share identical histories, retrieved memory, and tool budget, the gap is explained by the ~ 400 -token difference in textual context. This demonstrates that TIMECLAW’s gains arise from context comprehension rather than distributional pattern matching on the history alone. Figure 11 overlays both forecasts on the shared past and ground truth.

Figure 11: Case study of TIMECLAW on a pair of CiK SVAR tasks that share their numerical history and ground truth but differ only in the textual description of the data-generating process. With the explicit SVAR equation in context, the agent iterates the recursion on the known X_0 step schedule and recovers the ground-truth ramp (sMAPE 0.44%). With only a qualitative parent list, it extrapolates a smooth low-amplitude drift and misses the ramp entirely (sMAPE 124%). The two annotated boxes inside the panel reproduce the exact context delta between the two prompts.

Case Study C.2: TIMECLAW on TSRBench: How Retrieved Memory Steer the Decision

Core Task. A multiple-choice causal-discovery question from TSRBench (causal_reasoning, Elbe-River-Flood). The agent is shown 64 days of runoff measurements from five river sensors and must pick the 5×5 binary adjacency matrix that captures the causal structure (columns = cause, rows = effect). Ground-truth answer is D: an upstream-to-downstream cascade in which small tributaries (rivers R77, R546, R1071) feed the large downstream river R176, which in turn feeds R175.

Prompt: See Appendix I for the full template.

Visualization: We provide Figure 10 to better illustrate this case study.

Memory Retrieval Stage.

1. **Two-stage retrieval.** TIMECLAW embeds the test prompt with text-embedding-3-small and selects the top- K_{text} records from the training bank by cosine, then re-ranks the survivors by $L2$ distance on the fingerprint of the loaded series. Then TIMECLAW retrieves $k=3$ records.
2. **Rule extraction via summarize_trajectory.** Each retrieved trajectory is compressed to a tool-call spine plus a context_to_action block written during training. One block of the retrieved action in memory reads verbatim: *“the mechanism mapping is upstream-to-downstream edges.”* This is the transferable decision rule that primes the test agent.
3. **Prompt injection.** The compressed references are concatenated as a REFERENCES FROM PRIOR TRAINING block in front of the test prompt, only analytic spine and decision rules.

Execution Stage.

1. **Targeted tool use.** Primed by the retrieved rule, the agent runs only the inspections needed to confirm direction: list_channels $\rightarrow$ series_overview $\rightarrow$ five compute_acf probes, one per river. It skips the exhaustive per-channel statistics, peak-finding, and periodicity-detection that the no-memory agent runs through.
2. **Mapping the rule onto the new river set.** The agent matches its own observations to the retrieved rule: the channels with mean flow $\sim 660\text{--}680$ (R175, R176) form the downstream sinks; the three with mean flow < 4 (R77, R546, R1071) are upstream tributaries. The rule then determines option D, in which the sink row R176 is densely populated with parents from the tributaries and the tributaries themselves are source rows.
3. **Answer realisation.** The agent emits the final letter <answer>D</answer>, with a brief justification citing the retrieved cascade rule.

Counterfactual: Same Series, No Memory. Re-running the identical task at $k=0$ removes only the REFERENCES block. Without memory, the agent resorts to brute-force inspection, issuing 22 tool calls across channel_stats, compute_acf, find_peaks, and detect_periodicity for every channel. It then selects option A, which encodes the physically reversed cascade (R176 caused by R175; R77, R546, and R1071 each caused by R176). Without the retrieved rule, the agent lacks an anchor for causal direction and defaults to a topology inconsistent with the observed lead-lag patterns.

Evaluation. On the same instance, TIMECLAW changes from **wrong (A)** at $k=0$ (without memory) to **correct (D)** at $k=3$ (with memory), while reducing the tool-call budget from 22 to 7 (3.1 $\times$ fewer calls). The retrieved memory provides a transferable *decision rule*. This demonstrates memory improves reasoning by carrying compact rules that redirect the agent’s search process, rather than by matching numerical histories alone.

Figure 12: Case study on a TSRBench river-flood causal-discovery instance

Case Study C.3: TIMECLAW on TSAIA: Finance Tools Evolved from Experienced Trajectories

Core Task. TSAIA’s finance split presents the agent with portfolio and market time-series panels and asks it to answer quantitative multiple-choice questions. At the beginning, the agent’s toolbox only contains generic time-series inspection tools. It does not include finance-specific primitives for portfolio evaluation or market-factor regression.

Prompt: See Appendix I for the full template.

Tool Evolution.

1. **Routine extraction via `summarize_trajectory`.** During training, three recurring analytical routines appear across successful trajectories: portfolio risk-adjusted return estimation, portfolio risk estimation, and market-factor regression. Each successful trajectory is summarized into a compact routine description that records the input conventions, intermediate computations, and final decision rule. These summaries reveal reusable computation patterns.
2. **Evolving finance-specific tools.** From these recurring routines, TIMECLAW evolves three finance-specific tool schemas and adds them to the agent’s reusable toolbox:
 - `portfolio_sharpe(channels, weights, risk_free, period_per_year)`: compute risk-adjusted return for each portfolio option and compare the returned ratios.
 - `portfolio_var(channels, weights, horizon= $H$ , alpha= $\alpha$ , method="parametric")`: compute portfolio risk using task metadata and compare the resulting losses across options.
 - `capm_regression(asset_channel, market_channel)`: estimate market-factor statistics $\{\alpha, \beta, r^2\}$ and select the option whose target statistic matches the returned value.

Evaluation. Across the finance test split, evolved tools improve both accuracy and tool-use efficiency, especially when manual arithmetic is unreliable. On MarketAB-beta tasks, the average number of `capm_regression` calls per task increases to 0.83, and accuracy rises by +16.7. On VaR tasks, accuracy improves by +12.0, while the average tool budget drops from 6.3 to 5.2.

Figure 13: Case study of TIMECLAW on TSAIA’s finance MC split. Starting from a generic time-series toolbox, TIMECLAW observes recurring successful training trajectories and evolves finance-specific tools for portfolio evaluation and market-factor regression. At test time, these evolved tools replace brittle manual arithmetic with direct executable routines, improving accuracy while reducing unnecessary exploratory calls.

Prompt Template B.1: General Skeleton

General Prompt. All TIMECLAW prompts decompose the input into a fixed sequence of optional slots: a retrieval prefix, a tool-use hint (when the agent is wired to the MCP analysis server), a benchmark-specific header, the task question, an inline data block (only when the data is not preloaded into the MCP server), an option list (for multiple-choice tasks), an output-format clause, and a training-mode suffix. Concrete builders fill or omit each slot according to the task type.

```
{retrieval_prefix}
{tools_hint}
{header}

{question_or_context}

{inline_data_block}

{options}

{output_format}

{training_suffix}
```

Template fields. {retrieval_prefix} is the optional “REFERENCES FROM PRIOR TRAINING” block (Fig. 16, left), inserted at test time when the memory bank returns nearest-neighbor trajectories. {tools_hint} reminds the agent which MCP analysis tools are available and the recommended inspection pattern. {header} carries benchmark-level metadata. {question_or_context} contains the task statement (TSRBench / TSAIA) or the textual context fields (CiK). {inline_data_block} renders the partial numerical series as text when needed, and is often suppressed, since the canonical data is preloaded into the MCP server. {options} is the lettered option list only for multiple-choice tasks. {output_format} pins the response schema. {training_suffix} is the optional extension (Fig. 16, right) that converts the prompt into a trajectory-collection prompt during memory-bank construction.

Figure 14: General prompt skeleton used in TIMECLAW. A small set of optional slots covers all benchmarks; each concrete builder fills the slots appropriate to its task type and tool-availability regime.

Prompt Template B.2: TSRBench

TSRBench Prompt. TSRBench questions come in three surface forms: (i) “perception” questions with an inline `<ts><ts/>` placeholder where the series should be substituted, (ii) multiple-choice questions with a choices field, and (iii) free-form questions. The same template handles all three by toggling the rendering of the series block and the options block; the answer schema is always a single `<answer>...</answer>` tag.

```
Domain: {domain}
Task type: {task}
Series names: {names}
```

The numerical time series for this question is already loaded into your analysis tool server. Inspect it with the available tools before answering. A typical pattern: (1) `list_channels()` / `series_overview()`; (2) `channel_stats` / `channel_values` / `compute_acf` / `detect_periodicity` / `find_peaks` on individual channels. Do NOT guess values you haven’t retrieved through tools.

```
Question:
{question}
```

(MC subtasks only)

```
Options:
A. {option_A}
B. {option_B}
...
```

Reason carefully, then output your final answer wrapped exactly like this on its own line:
`<answer>X</answer>`.

Figure 15: Specific TSRBench prompt template. For Context-is-Key and TSAIA prompt template, please refer to our code implementation.

Prompt Template B.3: Memory-Retrieval Prefix and Training-Mode Suffix

Augmentation blocks. Two cross-cutting blocks compose with every builder above. The *retrieval prefix* is inserted at test time when the memory bank returns nearest-neighbor trajectories; the *training suffix* is appended at memory-bank-construction time so the agent produces a justifying analytic trace plus a transferable `<context_to_action>` explanation.

Retrieval prefix (test time, when $k > 0$ neighbors are retrieved):

=== REFERENCES FROM PRIOR TRAINING ===

Below are similar prior tasks (same task family where possible) and how they were solved, including which tools were called, the key numbers returned, and the final answer. Use them as a guide for the analytic process. The correct answer for the current task may differ; do not copy blindly.

Reference 1:

```
[family=<family_key>]
<trainer's transferable explanation>
tool_name(args) → <truncated_response>
tool_name(args) → <truncated_response>
...
```

Reference 2: ...

=== END REFERENCES ===

Training-mode suffix (memory-bank construction):

TRAINING MODE

Do TWO things before emitting your final answer:

1. Inspect the series with whatever tools are available and walk through the reasoning step by step.
2. Output a `<context_to_action>...</context_to_action>` block (≤ 3 sentences) that explicitly states:
 - which sentences in the context (background / scenario / constraints / question / options) drive the answer's shape, AND
 - the rule that maps those sentences to that shape (e.g. "scenario says 'heat wave for 2 hours' → ground truth has a $4\times$ spike at the stated start → because air-conditioning load scales with cooling demand").

Make this block transferable, NOT series-specific. Future test-time agents will read it to map their own context, so avoid statements like "the mean is 850" and prefer statements like "scenario X implies pattern Y because of mechanism Z".

Then produce your final answer in the format requested above.

Figure 16: The retrieval prefix (top) surfaces the analytic spine of nearest-neighbor trajectories from the memory bank. The training-mode suffix (bottom) is applied during the memory bank construction, so the trainer agent produces the transferable explanation block that the retrieval prefix later surfaces.